Diplomarbeit

Behaviour-Aware Load Balancing for a Micro-Kernel

Philipp Eppelt

18. Februar 2016

Technische Universität Dresden
Fakultät Informatik
Institut für Systemarchitektur
Professur Betriebssysteme

Betreuender Hochschullehrer: Prof. Dr. rer. nat. Hermann Härtig
Betreuender Mitarbeiter: Dr.-Ing. Adam Lackorzynski

Aufgabenstellung für die Diplomarbeit

Design & Implementierung eines Lastverteilungssystems für Mehrprozessor-Mikrokernsysteme

Aktuell unterstützt L4Re keine dynamische Lastverteilung auf mehrere Prozessorkerne. Ein Thread muss dem Scheduler selbst mitteilen auf welchem Kern er ausgeführt werden will, ansonsten wird er auf Kern #0 ausgeführt.

Um Mehrprozessorsysteme besser auszulasten kann eine Lastverteilungskomponente eingesetzt werden, die Threads selbstständig auf mehrere Prozessoren verteilt. Hierbei können verschiedene Vorgehensweisen zum Eisatz kommen. Das L4Re-System hat Ausführungsanforderungen bezüglich Sicherheit sowie Echtzeit, die durch Isolation von Subsystemen auf unterschiedlichen Prozessoren umgesetzt werden müssen. Einzelne Applikationen haben aber auch die Anforderung mehrere Threads prozessorlokal auszuführen. Der Energieverbrauch eines Gesamtsystems ist ein weiterer Gesichtspunkt und kann durch eine Lastverteilungskomponente beeinflusst werden, indem nicht benötigte Prozessoren potentiell abgeschaltet werden. Die Lastverteilungskomponente muss hierbei für entsprechend leere Prozessoren sorgen.

Ziel der Arbeit ist es, die Informationen zu identifizieren und Mechanismen bereitzustellen, anhand derer eine Lastverteilungskomponente mit ihren Algorithmen Entscheidungen treffen kann. Dabei muss im Zusammenspiel zwischen Anwendungen und Betriebssystem die Last des Systems analysiert, eventuell zukünftige Last vorausgesagt und anhand von Einstellungen Lastbalancierungsentscheidungen getroffen werden. Die Lastverteilungskomponente soll in das bestehende L4Re-System integriert werden. Für die Lastvorhersage sollen ausgewählte Heuristiken benutzt, aber auch Sicherheitskriterien beachtet werden. Die Heuristiken sollen anhand verwandter Arbeiten ermittelt werden.

Die Lösung soll exemplarisch umgesetzt und mit einem multi-threaded Benchmark evaluiert werden. Dabei sollen mindestens zwei Lastbalancierungsalgorithmen zum Einsatz kommen: ein Energieoptimierender und ein Latenzoptimierender sowie soll die Ausführung von sicherheits- und echtzeitkritischen Applikationen beachtet werden.

Die zentrale Fragestellung ist welche Informationen lassen sich effizient durch Nutzermechanismen sammeln und welche sind nötig, jedoch nur im Betriebssystemkern vorhanden. Hierbei sollen entsprechende Schnittstellen vorgeschlagen werden, die außerdem Sicherheits- und Effizienzkriterien unterliegen.

Erklärung

Hiermit erkläre ich, dass ich diese Arbeit selbstständig erstellt und keine anderen als die angegebenen Hilfsmittel benutzt habe.

Dresden, den 19. Februar 2016

Philipp Eppelt

Contents

1 Introduction **1**

2 Technical Background **5**
2.1 Terminology 5
2.2 Related Work 7
2.3 Linux Completely Fair Scheduler 14
2.4 Fiasco.OC & L4Re 15
2.5 Intel Haswell Architecture 16
2.6 Summary 17

3 Design **19**
3.1 Hyper-Threading 19
3.2 Isolation 20
3.3 Communication 21
3.4 Energy Model 24
3.5 Load Balancer 25
3.6 Load and Balance 28
3.7 Summary 29

4 Implementation **31**
4.1 Kernel Performance Counter 31
4.2 Task Configuration 33
4.3 Internal Data Management 36
4.4 Algorithms 37
4.5 Summary 39

5 Evaluation **41**
5.1 Benchmark and Measurement Setup 41
5.2 Measurements 43
5.3 Summary 52

6 Future Work **53**

Bibliography **57**

List of Figures

1.1 Processor and memory performance gap 1980 - 2010. 1
1.2 Size and access latencies for different levels of the memory hierarchy. The numbers displayed are from a Nehalem generation Xeon™ server CPU. Figure from [HPA12, p. 72]. 2

2.1 Intel Haswell generation cache inclusiveness. 14
2.2 Schematic of cache sharing across cores of a Haswell generation processor. . . 17

3.1 Load balancer interaction with clients. 26
3.2 MPDE Cycle with topology awareness. 27

5.1 Solo run-times visualized using one box plot for each balancing algorithm; STB and TB use a 100ms balancing interval. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. 45
5.2 Solo run-times visualized using one box plot for each balancing algorithm; STB and TB use a 10ms balancing interval. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. 46
5.3 Concurrent run-times visualized using one box plot for each balancing algorithm; STB and TB use a 100ms balancing interval and all time measurements are taken from the same concurrent run. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. 48
5.4 Concurrent run-times visualized using one box plot for each balancing algorithm; STB and TB use a 10ms balancing interval and all time measurements are taken from the same concurrent run. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. 49
5.5 Concurrent run-times visualized using one box plot for each balancing algorithm; from top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. Time measurements are taken from the same concurrent run. 50
5.6 Visualized difference between the balancing algorithms per application and overall; for normal load on the left and high load on the right. 51

List of Tables

5.1 Average increase in run-time per application and overall; for normal load on the left and high load on the right. 51

List of Listings

4.1 User land interface for retrieving and resetting a thread's performance counters; UTCB-pointer left out for brevity. 32
4.2 Implemented set of configuration parameters for priority range, isolation, and communication groups. 33
4.3 Task creation with create call to the load balancer factory and replacement of the default scheduler in the Ned startup script. 34
4.4 Example of the a benchmark startup using the load balancer as scheduler. . . . 34
4.5 Implementation of the configuration type stored by the task proxy. 35

1 Introduction

Starting with the common availability of computer systems in the 1980's, computing and memory facilities have developed in different directions. Uniprocessor speeds increased until they hit the frequency wall, which then led to the rise of multi-core processors. Whereas memory sizes increased significantly, memory speed did not see an equal development. The increasing difference in processing speed, displayed in figure 1.1, led the processor vendors to obscure the gap by introducing numerous levels of on-die caches. These caches allow to prefetch memory based on typical memory access patterns and reduce the number of times the CPU sits idle. However, search duration increases with cache size and if the requested data is not present, the cache-search duration increases the memory access latency additionally. Figure 1.2 shows size and latency of a Nehalem generation Intel CPU.

Besides reducing the number of waits for data from memory, the waiting time can be used for computation. Out-of-order processors execute other instructions while waiting for data and therefore reduce the idle time of the processor. Another technology to use idle processing resources is Simultaneous Multi-Threading (SMT). It employs several hardware threads sharing a processor's execution units to provide several instruction streams. While one instruction stream waits for memory, others are able to utilize the otherwise idle resources.

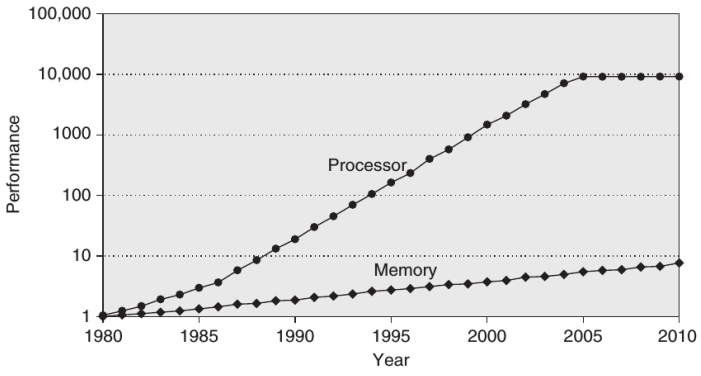


Figure 1.1: CPU and memory performance development since 1980. CPU performance is plotted as memory request per second, memory performance as requests served per second. Figure from [HPA12, p. 73].

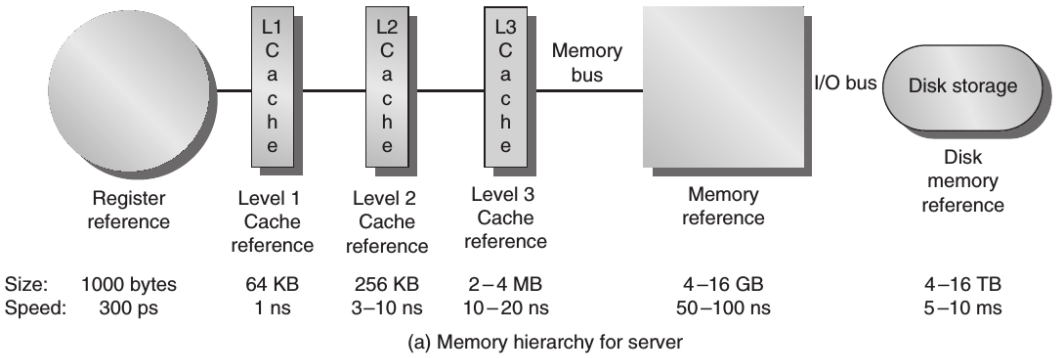


Figure 1.2: Size and access latencies for different levels of the memory hierarchy. The numbers displayed are from a Nehalem generation Xeon™ server CPU. Figure from [HPA12, p. 72].

Besides hardware solutions, software can also increase memory efficiency. Programmers of performance-critical software invest time and effort to adjust their program's data to fit into cache lines and allow for efficient prefetching. However, these custom adjustments are limited by the environment the software executes in. Two memory intensive threads sharing a processor will evict each other's cache lines, thus decreasing each other's performance as they need to fetch the data from memory again. In the era of multi-cores, to decide if these two programs will run on the same core and cache is the challenge the load balancer faces.

A specialized load balancer can take offline knowledge into account that is not accessible by the kernel; for example offline analysis of a program's cache usage. Such a specialized load balancer can improve the run-time of its clients by improving cache utilization. A general-purpose load balancer does not know its client tasks beforehand, thus limiting the amount of information available. Besides performance improvements, the load balancer can also satisfy isolation requirements of client applications; for example by assigning a security critical application an exclusive core to mitigate side-channel attacks.

This thesis explores possibilities to gather run-time information about the behaviour of a load balancer's clients, to provide for efficient execution of all clients. Besides execution time requirements, the load balancer should account for cache usage, communication patterns, and isolation requirements. I discuss various sources for these information with their merits and flaws and implement a prototype to show the benefits of load balancing that is aware of client behaviour.

Chapter 2 defines terminology for this work and presents previous publications on SMT, shared resource congestion, and communication awareness. It also presents Linux's completely fair scheduler, Fiasco.OC and L4Re, and properties of the CPU architecture the prototype runs on. After presenting the current state of research, chapter 3 discusses the building blocks for the load balancer: SMT, isolation needs, sources for communication behaviour, an energy consumption model, and the architecture of the load balancer itself. Chapter 4 presents implementation details and implemented algorithms. Subsequently, chapter 5 evaluates the prototype and answers the question, how beneficial the balancing is. Chapter 6 motivates further research on this topic and concludes this thesis.

2 Technical Background

In this chapter I provide an overview over terminology and techniques I use during this thesis. I begin with an introduction to basic terminology in section 2.1.

Different information sources can enhance load balancing decisions regarding SMT performance, resource usage, and communication relationships. Besides these three performance oriented topics, a load balancer can also improve isolation for security critical tasks and consider overall energy usage. I present state of the art research from these areas in section 2.2.

In section 2.3, I give an overview over Linux's completely fair scheduler. I discuss benefits and drawbacks of Fiasco.OC and L4Re in section 2.4, before I introduce the targeted CPU architecture in section 2.5.

2.1 Terminology

In the following I make the reader familiar with the terminology of the field. Readers may continue with section 2.2 and use this section as reference, if a term is ambiguous or unknown.

User Thread Control Block (UTCB) A UTCB is an user-level memory section containing information about a thread, the thread's communication registers and communication buffers. It is primarily used for message transfer.

Scheduling Context (SC) A SC contains information for the scheduler consisting of a priority level, execution time, remaining time quantum, and core afffinity mask.

Thread A thread is a unit of execution, consisting of a kernel and UTCB, a SC, and further interfaces.

Task A task is a unit of isolation. It manages different address spaces; for example memory, capability, and I/O port spaces. Threads run within tasks, but tasks do not necessarily contain a thread.

Capability A capability is a rights management concept. The system allows access to an object only if the caller provides a capability containing appropriate rights. It is orthogonal to access control lists, where the object knows, who may access it.

L4 Inter-Process Communication (L4-IPC) L4-IPC is one way for threads in L4Re/Fiasco.OC to communicate with one another. It is synchronous communication with the following modes of operation: send, open wait, closed wait, reply & open wait, and call. A thread initiating an open wait, waits for an arbitrary thread to contact him, whereas a closed wait, expects a communication intent of a specific thread. A call is normally used for communication with a service, where a reply is awaited, hence it combines a send and a closed wait operation. To initiate an L4-IPC in L4Re/Fiasco.OC the sending thread needs to hold a capability on a channel going to its communication partner.

Instructions Per Cycle (IPC) IPC describe the number of instructions executed per processor cycle.

Core A physical CPU core is referred to as core.

Cache Generally, a CPU cache is used to store frequently accessed data close to the CPU core to reduce access times. As a cache has size restrictions, depending on the type of cache different evocation policies can be applied, e.g. First-In-First-Out (FIFO) or Least-Recently-Used (LRU). There are multiple levels of caches in modern CPUs, creating a hierarchy; three levels are common. If modified data is evicted from the highest level of the hierarchy, it is written back to memory.

Cache Coherence Data must be kept consistent between different CPU cache levels to provide a strict order of writes to the same memory relocation. Multi-socket x86 system employ a snooping protocol to keep caches coherent. Each cache broadcasts every update over the address bus and all other caches update the status of their copy of the data. Haswell generation processors use the MESI protocol to keep their data coherent ([Int15b, pp. 2-22]).

Migration The process of moving a thread from one core to another core is called migration. The cores do not necessarily need to be on the same chip, nor do they need to be in the same machine. Only the thread is migrated, cached data remains on the old core.

Execution Unit (EU) EU refers to hardware components inside a processor core which perform a computation; for example integer or floating-point units.

Hardware Thread A hardware thread describes an architectural state which consists among others, general-purpose, control, and machine state registers. The operating system multiplexes the software threads to the hardware thread. Only a hardware thread can access EUs of the processor.

Simultaneous Muli-Threading (SMT) SMT is a technique to multiplex several hardware threads on the same physical core. Thus, sharing all the core's resources – integer and floating point units, caches, prefetcher, etc. – among the core's hardware threads. The goal is to improve the utilization of the EUs by increasing the number of instructions available to them. SMT transforms thread level parallelism to instruction level parallelism. I reference a SMT-core as logical core.

Hyper-Threading (HT) HT is the Intel® specific implementation of the SMT-concept.

Chip Multiprocessor (CMP) & Symmetric Multiprocessor (SMP) A SMP architecture describes a set of completely equal physical cores sharing the Last-Level Cache (LLC). In contrast, CMP architectures consist of two equal processors soldered on one die, thus sharing nothing. In both cases, all cores follow a cache-coherency protocol.

Instruction Set Architecture (ISA) ISA describes the set of hardware instructions executed by a processor.

Intervall Describes a pre-defined amount of time.

Wall clock time Describes the time passed in real life.

Execution time Describes the time a thread spends executing on any core.

Turnaround time Time between start of a thread and its completion.

2.2 Related Work

In this section I present prior work and emphasise the parts pertinent to this thesis. One assumption for this work is already in place: the target architecture is a single ISA, partly symmetric homogeneous multi-core. Partly symmetric means the target architecture supports SMT, but besides this, it is an SMP architecture. Hence, research proposing hardware changes is out of scope; such as [Cru+14], where cache coherence messages are monitored to increase a hardware communication counter. Also, research focusing on heterogeneous or multi-ISA architectures is not part of this review; for example in [Sar+15] the authors compare different core categories regarding expected performance of the current workload.

I will focus this review on research regarding five topics: SMT, resource congestion, communication, energy usage, and security. A load balancer must address all these topics, to efficiently use available resources and to make repeated execution more predictable.

Simultaneous Multi-Threading. Snavely and Tullsen introduce the term *symbiotic scheduling*, describing a scheduling technique for SMT-systems in [ST00]. They propose scheduling those two threads on a shared core, which provide the best utilization of said core. The combination of two threads on two corresponding SMT-cores is called a co-schedule. A co-schedule's performance is defined as the turnaround time.

In their experiments they run independent threads on a simulator modeling a Compaq Alpha 21264 out-of-order processor with hardware additions for SMT. This simulator model includes performance counters to "capture dynamic execution information"([ST00, p. 236]). To determine a good schedule, the authors compute the weighted speedup *WS* for a fixed time interval. For each thread the change in instructions per cycle (IPC) compared to single-threaded execution is calculated. The sum of these values for all threads is the weighted speedup of this schedule.

$$\mathrm{WS}(t) = \sum_{i=1}^{n} (\text{realized IPC thread}_i / \text{single-threaded IPC thread}_i)$$

Their developed scheduler consists of two phases: sampling and symbiosis. The sampling phase measures different thread combinations and moves to the symbiosis phase, when enough samples were gathered. During the symbiosis phase, the scheduler selects the schedule, which promises the highest weighted speedup. As the thread mix changes, the scheduler will move back and forth between these two phases.

A sampling phase measure describes the performance of the threads in a co-schedule, as measured by performance counters. The predictors use these measures of each single thread to predict the performance of a future co-schedule. Among others, the authors created predictors for the following: data cache hit rate, floating point unit conflicts, and floating point queue conflicts. However, the best predictor in their study made a majority vote of all other predictors to determine the best schedule. Snavely and Tullsen conclude, that good thread symbiosis depends not only on high IPC, but also on usage conflicts in shared hardware resources or cache hit rates.

Still, measuring performance improvements due to increased symbiosis is difficult. While Snavely and Tullsen claim a reduction in average turnaround time by up to 17% in [ST00], Eyerman et al. state a mere 3% to 6% increase in average throughput with an optimal symbiotic scheduler compared to a first-come-first-serve scheduler. The authors explain the notable difference in [EMR15] as different experimental views. Snavely and Tullsen study a fully loaded system with twice as much ready threads as SMT-cores and observe the turnaround time. In this scenario, the time the thread spends waiting in the ready queue causes the largest increase in turnaround time. A slight increase in maximum throughput reduces the waiting time and consequently leads to a significant decrease in turnaround time. Also, reordering the threads in a shortest remaining time first manner can reduce the average turnaround time as well. For

these reasons Eyerman et al. make the case that turnaround time alone can be a misleading measure.

Also, maximum throughput cannot be used, because systems are not designed to run at peak load all the time, as this leads to long queues and long turnaround times. If the system is not fully loaded, average throughput equals average load, therefore, this too is a useless comparator. Instead, they advocate to evaluate processor utilization — understood as average number of executing cores — and empty periods. If throughput improves, the processor utilization decreases and empty periods expand, as the time to execute the current load is reduced. Hence, using these two measures, improvements caused by the scheduler are comparable.

Although no large performance improvements can be expected, symbiotic scheduling proves to be relevant to improve resource usage. However, if no offline knowledge is available, it is difficult to connect performance gains to scheduling decisions. As modern Intel® multi-cores use HT in each physical core, symbiotic scheduling provides an opportunity for performance gains. Together with hardware peformance counters, it might be possible to at least avoid bad co-schedules.

Resource Congestion. A different technique is presented by Banikazemi et al. regarding thread-to-core mappings on a chip multiprocessor system. In [BPA08] they present PAM: a performance/power-aware meta-scheduler, working on top of the default linux scheduler. While the default scheduler runs, PAM gathers information via performance counters on cache occupancy, cycles per instruction, and the miss ratio of L1 and L2 caches for each thread. If PAM can predict a better thread-to-core mapping based on this information, it overrules the default scheduler and enforces the better mapping. PAM uses a *goodness* metric to rate the mapping of the last time interval. If the goodness value of the PAM mapping is better than the last observed default mapping, PAM continues to overrule the default scheduler. If it is worse, PAM falls back to observing the default scheduler. The goodness metric can describe performance, power or energy consumption.

Their experiments run on a dual socket machine with one quad-core per socket, where two cores of each quad-core share an L2 cache. The processor does not support SMT. Hence, the performance gains stem not from symbiosis, but rather from congestion awareness.

Becoming aware of and reducing cache congestion is also topic of [Kna+08]. Like Banikazemi et al., they work on a CMP-machine with a shared L2 cache per chip. They try to spread memory intensive tasks as much as possible, to reduce interference in the shared cache. The authors introduce the metric called *cache weight*, which denotes the LLC misses per cycle for the last

time interval. Their research shows that longer histories worsen the prediction, as thread behaviour changes over time. To achieve the best possible spread of all threads across cache groups, Knauerhase et al. balance the threads in the LLC group based on the aggregate cache weight. New threads are assigned to the cache group with the lowest weight and only cache light threads are migrated across group boundaries. However, to care for workload changes, their scheduler periodically migrates an overweight thread to the lightest LLC group.

Furthermore, cache light threads suffer next to cache intensive ones, as they most likely start with a cold cache each time they are scheduled. To compensate for this, Knauerhase et al. propose to transfer CPU time from the cache-heavy to the suffering threads. In their experiments, this fairness compensation reduces the performance degradation of the most suffering thread to less than 1%.

Both, CMP and SMP systems share more resources than just the cache. In [FBZ10], [ZBF10], and [Zhu+12] Fedorova et al. and Zhuravlev et al. further investigate congestion on shared resources. Although their assumption in [FBZ10] is that memory reuse patterns are the indicator of choice to predict cache interference between threads, their research leads to the conclusion that LLC misses are a superior predictor. Memory-reuse patterns model the access and miss frequency to a memory area for a particular application. This analysis is to be done offline and assumes that applications with high reuse and low miss rates do not affect other applications negatively. But applications with low reuse and high miss rates are likely to evict cache lines of others.

Zhuravlev et al. use this analysis and develop a metric called *pain*, which consists of two parts: sensitivity and intensity. The former describes how much an application suffers from sharing caches with others, the latter how much an application will hurt others. The pain application A feels, when scheduled with B, is the product of A's suffering and B's intensity. Vice versa for B. The sum of both describes the expected performance degradation for these two applications, when sharing the same LLC.

To approximate sensitivity and intensity online, they assume cache miss and access rates to be a good measure, but it turns out, only the cache miss rate is relevant. In contrast to memory-reuse patterns, which focus solely on the cache as shared resource, the cache miss rate also displays the load on prefetching hardware, memory controller and memory bus. Load on these shared resources turnes out to be the main reason for performance degradation in most of their benchmarks ([ZBF10]).

Zhuravlev et al. implemente their off- and online derived pain metrics in two scheduling algorithms: Distributed Intensity (DI) and Distributed Intensity Online (DIO). DI uses the pain metric derived offline from the memory-reuse patterns, whereas DIO uses *approx-pain* derived

from the LLC miss rate. Like [BPA08], both schedulers are implemented in the Linux user level and override the default scheduler.

The DI algorithm sorts the applications based on their solo cache miss rate derived offline. This order is used to split the application set into two halfs: above and below average miss rate. Each core runs all threads of one application from each half. If the miss rate changes at run-time, the order changes, which could impair the performance. However, Zhuravlev et al. observe only minor place switching, which they do not expect to harm the performance.

DIO on the other hand, constantly measures the applications miss rate and is therefore able to react to phase changes during run-time. The algorithm collects data every 10^9 cycles and uses a running average to sort the applications. Besides this it acts like DI.

Zhuravlev et al. run tests on a Intel® Xeon and AMD Opteron. The Xeon consists of eight cores on four chips sharing the L2 cache. The Opteron eight core has two chips sharing an L3 cache each. Leading to four cache groups in the Xeon case and two in the Opteron case. Their tests show only small performance differences for their tested workloads on both machines. The authors run eight different workloads on each machine and compare the performance improvement to the default Linux scheduler. Of the total of 16 tests only three tests show performance differences above two percent between DI and DIO. In the Opteron's case DIO improves all workloads, whereas DI shows gains in all but one. The Xeon shows a mixed picture for DI and DIO: from 14 percent gains to four percent loss compared to Linux's default scheduler. According to Zhuravlev et al., the good overall performance of the default scheduler can be accounted to the small test set and the relatively low probability to select the worst thread combination. They emphasise the performance gain of each individual application with DI and DIO scheduling and the reduced variance in execution time from run to run.

The test workloads consist of independent applications. Thus, the trade-off between placing two communicating, high miss rate applications on the same core, or communicating across chip boundaries was no research question. Additionally, CMP architectures gave way to SMP architectures, which do not necessarily provide strictly separated cache groups. Therefore, the performance gain must be reevaluated.

Communication. Up to now, all research covered assumes independent, non-interacting threads. Hofmeyr et al. present in [HIB10] a load balancing technique explicitly assuming communicating, parallel applications. The parallel application model consists of one application spawning several worker threads, where a computation interval is followed by a communication barrier. They propose a technique called *speed balancing* with a notion for fast and slow cores. Their concept assumes the number of threads N to be greater than the number of cores C.

Consequently, the number of threads per core will be either $T = \lfloor N/C \rfloor$ or $T + 1$. The authors call cores running T threads fast cores and cores running $T + 1$ threads slow cores, respectively.

For their implementation, they also assume to be running alongside other workloads on the same hardware. Therefore, the term *speed* is defined as the relation of execution time per elapsed wall clock time. The authors' algorithm runs one observation thread per core measuring the speed of each thread and of the core, respectively. The sum of all core speeds determines the global average speed, which distinguishes fast from slow cores. To provide equal progress opportunities, each thread needs to run on a fast core at least once. So, the observation thread on a fast core searches for a suitable thread on a slow core. Additionally, the observation thread tries to migrate the least migrated thread to avoid constant migration of the same thread.

Their implementation takes an application as input, starts it and distributes its worker threads. They do neither model communication between different tasks in a client-server notion, nor at arbitrary points in time. Hence, reducing the time the slowest thread needs to reach the synchronization point improves overall application performance.

Energy efficiency. Performance is not a one dimensional value anymore. Performance per energy is the important measure on battery supported devices like notebooks or smartphones, and also in datacenters, as less energy usage mean less energy cost for running and cooling. In [Ime+15] the authors compare a mobile Intel® Haswell and a Samsung Exynos5 Octa regarding their minimal energy consumption. The authors show that different strategies are necessary to achieve minimal energy consumption for each of the two processors. In case of the Intel® Haswell processor, race to idle proves to be the strategy of choice to reduce energy consumption. The Samsung processor on the other hand, should run constantly at low clock speeds.

[LH11] considers systems loads below 30% and different workloads: video playback and a web-server. Their measurements suggest, that for x86 processors it is best to maintain low clock speeds, as long as the system load and quality of service guarantees allow it. During idle time, the processor changes to deep sleep states and saves energy. The recovery from deep sleep states – power up core, load data into core local caches – increases system load only slightly. To conserve energy, *turbo boost* – raising the frequency of one core above standard, if other cores are idle – should be avoided, as the gained frequency comes at a high energy cost.

It is interesting for this work, how a load balancer can change the assignment of threads to cores, to reduce overall energy consumption. In section 3.4 different models will be discussed.

Cache side-channel attacks. The CPU cache proved to be a major side channel to deduce private cryptographic keys. Two different attacks are discussed here: FLUSH+RELOAD and PRIME+PROBE.

Three conditions are necessary for a FLUSH+RELOAD attack: the x86 instruction to flush a specified cache line, if read access is allowed; shared memory between attacker and victim, e.g. binary code in shared libraries or executables; and conditional branches in the code path of the attacked program. Yarom and Benger present in [YB14] an attack to recover private keys via FLUSH+RELOAD. To that end, they flush a set of cache lines from the cache and reload it after a waiting period. If the cache line was present in the cache, the time it takes to read the data is small; reading the data from memory is significantly slower.

The attack uses the fact that the OpenSSL implementation of the Montgomery ladder[1] executes a different branch, depending on the value of the current bit in the private key. If the data containing the *then* branch is present in the cache, the tested bit was one, if the *else* branch data is present, the bit was zero. The length of the waiting period – between flushing and reloading the chache – presents a trade-off between missing subsequent accesses, if the period was too long, or observing single accesses as duplicates of themselves. In [YB14] the private key was recovered using this technique. Other implementations of the Montgomery ladder, e.g. in Network and Cryptographic Library (NaCL)[2] by Bernstein et al. do not rely on branching and are therefore not susceptible to this attack.

However, shared memory with the target application is necessary. Since no data is shared between virtual machines FLUSH+RELOAD is impossible. The PRIME+PROBE attack on the other hand works without this requirement. This attack writes known data to a set of cache lines, the eviction set, and accesses the same data after a waiting period. If a cache line was used by other processes the data is gone and it takes longer to reload it from memory.

The attack utilises the inclusiveness property of the CPU's LLC, to observe changes in core local L1 and L2 caches. The inclusive LLC contains each cache line present in any of the L1 and L2 caches, as shown in figure 2.1.

Therefore, the eviction set must cover the cache lines in the LLC that correspond to the core local caches. Hence, the core running the application under attack must be known.

After the right set of cache lines is found, the access frequency can be observed. The access to the cache set reveals squaring operations in the algorithm. Based on the time passed between two squaring operations, the attacker deduces a one or zero bit. The execution path in case of a one bit is longer, hence more time passed. Two attacks are presented in [Liu+15] and [Inc+15], both recovering a private key of an attacked virtual machine. Inci et al. test their attack on Amazon EC2 instances, proving its practicality in a real world scenario.

[1] fix published 05 Jun 2014, see http://www.openssl.org/news/secadv/20140605.txt
[2] https://nacl.cr.yp.to

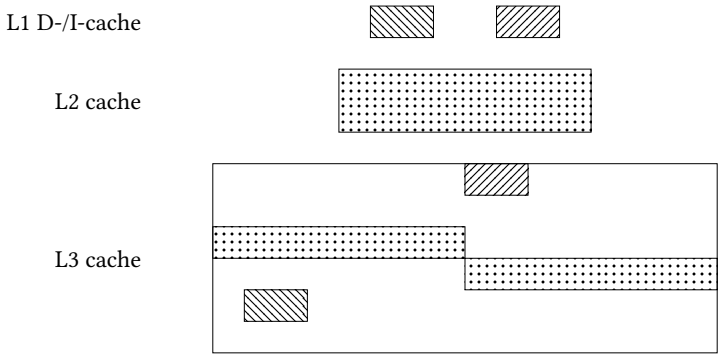


Figure 2.1: The LLC of Intel® Haswell generation processores are inclusive. Each core contains 32KB D-, 32KB I-, 256KB L2, and 1.5MB L3 cache

Separating security critical applications on architectures with shared LLC raises the difficulty of an attack, but does not mitigate it. Only a physical separation of the caches or access right management prevents a PRIME+PROBE attack.

2.3 Linux Completely Fair Scheduler

Since Linux version 2.6.23, Linux uses the Completely Fair Scheduler (CFS) ([SMa08]). It aims at complete fairness, in regard to execution time provided to each task. To achieve fairness, it measures so called virtual run-time of each task and selects the task with the lowest account to run next. Internally, CFS uses a red-black tree to sort the tasks regarding virtual run-time.

CFS also supports group scheduling to provide fairness between tasks with one thread and tasks with many threads. If a task spawns many threads, they all execute on the account of the main task, preventing one task to acquire more execution time than others.

Conceptually, CFS runs the currently selected task until the virtual run-time is higher than the virtual run-time of any other task. Then a task switch occurs. As this would lead to constant task switching, two measures are used to counteract this: *target scheduling latency* (TSL) and *minimum task run-time granularity*.

The target scheduling latency describes an amount of time shared between a number of tasks. If two tasks are runnable, and the TSL is 20ms, then each task runs for 10ms. However, if more tasks become runnable, the amount of time per task shrinks and eventually the switching overhead is higher than the time a task spends executing. To prevent this situation, the minimum task run-time granularity defines a minimum amount of time a task needs to execute. As the number of tasks grows, the TSL increases to be able to schedule each runnable task once for the minimum task run-time.

Priorities are accommodated by weighting the virtual run-time accounted to each task depending on the priority level. The virtual run-time of a high priority task increases slower than the virtual run-time of a low priority task, leading to more actual execution time for the high priority task.

Domains, Groups, & Load Balancing. In CFS a domain represents a unit of physical hardware; for example, a physical core is a domain and the two hyper-threads running inside constitute a group each within this domain. On each level of the hierarchy, the current level is the domain and the lower hierarchy levels are groups. Each group within a domain is equivalent to other groups in the same domain ([Cor04]).

CFS does active load balancing only between groups in the same domain. Their load is compared and if imbalances are found, CFS tries to migrate a task from one group to another to establish balance again. A cache affinity flag indicates, if the task ran recently and might still have valid cache lines. If the cache affinity flag is set, the task is not considered for migration.

2.4 Fiasco.OC & L4Re

The related research I present in section 2.2 states the following three feature requirements: hardware performance counters, assignment of threads to specific cores, and the ability to overrule the kernel scheduler.

Fiasco.OC is a µ-kernel of the L4-family and follows the design paradigm to only implement mechanisms in the kernel and no policies. For performance reasons, the scheduling policy is the only policy implemented in the kernel. The scheduling parameters in a scheduling request from user land consist of a priority level, a time slice size request and a descriptor stating on which cores the thread wants to be scheduled. The kernel schedules the thread on the first available core in the descriptor, checks if the priority level is in a valid range, and assigns a time quantum. It does neither balance the amount of threads per core, nor prevent oversubscription. However, it provides the mechanism to design a more sophisticated load distribution scheme as a user-level service.

OC in Fiasco.OC stands for object capability, a system providing a thread local namespace for references to other objects, such as the scheduler, object factory, and memory management. A new task populates its capability space with a set of capabilities inherited from its parent to be able to interact with the system. This set is completely independent of its parent capability namespace; parent and child can replace capabilities in their capability space without affecting the other. This property enables the use of proxies, implementing the same interface, but a different policy on top of the base service. An object registers the new service in its capability space. So different services can be implemented on top of each other, providing different policies for subhierarchies of the object space.

The L4 runtime environment provides a scheduling service skeleton, which defines the kernel and user interface. Using this skeleton and the capability namespace property, a scheduling service can replace the default L4Re scheduler and implement further analysis of each thread. The

skeleton also enables thread-specific proxies with an individual configuration. This mechanism will be used in this work.

Regarding hardware performance counters, the kernel provides neither a system call interface to read or configure them, nor per thread accounting of configured counters. Access to hardware performance counters is critical for this project, so I need to implement a mechanism to provide this for a user-level service.

2.5 Intel Haswell Architecture

In section 2.2, I mention several hardware features. The most important of all are performance counters to measure cache misses, cycles and instructions executed. The performance counters in the target processor — an Intel® Haswell generation Core i7-4660K – are able to count these three events and many more. The processor contains Architectural Performance Monitoring version 3 capabilities, which consist, among other features, of three fixed-function performance counters counting Instruction Retired, Unhalted Core Cycles, and Reference Instruction Retired ([Int15c]). In contrast to the first two counters, the last counter is unaffected by processor speed changes due to *dynamic voltage and frequency scaling* or *turbo boost* features.

Besides these, four general purpose performance counters are available per logical core, which can be programmed to count one specific event. All Intel Core i generations support architectural performance monitoring in different versions, but all versions are guaranteed to support the following events: Unhalted Core Cycles, Instruction Retired, Unhalted Reference Cycles, LLC Reference, LLC Misses, Branch Instruction Retired, and Branch Misses Retired ([Int15c]).

Besides these central events, each hardware generation supports a different set of so-called "non-architectural performance events". This non-architectural event set allows to monitor many specific events, e.g. L2-misses/-hits, micro-ops per logical core executed per cycle, or unhalted core cycles executed in ring 0.

Another difference to the target processor, compared to most processors used in earlier research, is the SMP architecture. In contrast to CMP, the architecture does not define cache groups, because the LLC is shared among all cores. Figure 2.2 shows an example of the cache hierarchy of the Intel Haswell processor. Each core consists of two HT cores, sharing L1- and L2-caches. All cores on one socket share the same LLC. The LLC is inclusive, meaning each cache line present in L1I-, L1D-, or L2-cache is also present in the LLC ([Int15b, pp. 2-23]). This eases the lookup of cache lines in other cores on the same socket. If the line is present in the LLC the cache-coherency status indicates, if and in which core's caches it is present.

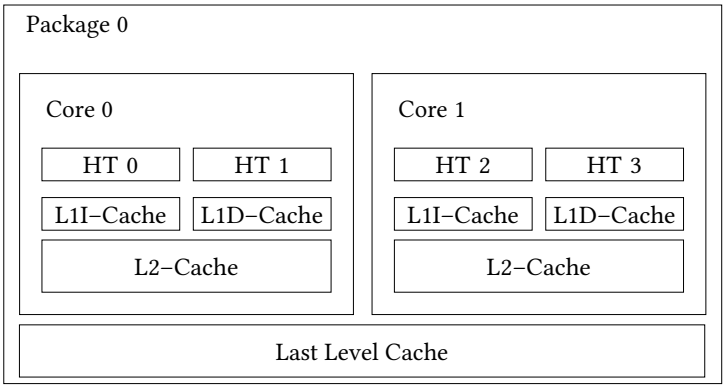


Figure 2.2:
Schematic layout of two cores of a Core i7-4660K quad-core. Each physical core has two logical cores sharing the core's L1I-, L1D-, and L2-cache.

Topology analysis. The topology of the CPU can be determined at run-time. It is useful to detect which two logical cores exist together on the same physical core, since Fiasco.OC boots the physical cores in a nondeterministic order. The CPU provides this information through the CPUID interface, if the extended topology enumeration leaf is supported ([Int15c, Vol.2A 3-179]). This is the case for the target CPU.

2.6 Summary

I presented separate approaches to improve different aspects of load balancing. Those aspects are SMT-aware balancing on shared cores, observation of congestion on shared resources, knowledge about communication relationships between tasks, and isolation requirements.

My goal in this thesis is to unite all these topics in a load balancing service for Fiasco.OC/L4Re. The load balancer should provide applications with security requirements with a separate core and cache, to reduce the attack surface for cache side-channel attacks. Furthermore, the load balancer increases application efficiency by considering communication relationships and differentiating between memory and compute-bound threads. As little offline knowledge should be used to provide the load balancer with necessary information regarding task properties and thread behaviour. Hardware performance counter play a vital role in this endeavour.

I implement the prototype service for one specific architecture, an Intel® Haswell Core i7-4660K.

3 Design

In this chapter, I describe the problems needing to be solved to achieve the overall goal of a behaviour-aware load balancer on the Fiasco.OC microkernel.

In general, a load balancer assigns the currently running set of threads to the available cores. To that end, it observers the execution time requirements of all threads and distributes the threads to all cores, such that each core processes an equal share of the total work. If the amount of work processed per core is uneven, the load balancer needs to reestablish balance by selecting a thread to migrate from an overloaded core to a less utilized core. The load balancer must account for the underlying architecture, as the performance improvement when migrating a thread from one SMT-core to its companion core, is likely smaller than migrating it to another physical core.

Besides execution time, cache affinity is also relevant to the load balancer. Migrating a thread which has a lot of present cache lines will hurt its performance, because the thread must load the necessary memory into the new core's cache, before it can continue execution.

In case of an imbalance the load balancer selects threads to migrate, after considering cache affinity and execution time requirements. A load balancing interval is usually much larger than a scheduling interval.

From chapter 2 four relevant areas for a behvaiour-aware load balancer can be deduced: SMT, isolation, communication and energy. After discussing these four areas, I explain the design of the load balancer service, before I finish the chapter with a discussion about load and balance.

3.1 Hyper-Threading

At the lowest level, a load balancer must balance the work of two corresponding HT cores. The performance gains of HT stem from the conversion of thread level parallelism to instruction level parallelism, by providing more instructions to the EU to fill their pipelines. However, if two threads issue the same group of instructions, no performance gain is possible, as they conflict in the usage of the same EUs. Hence, two threads with good HT performance use different EUs.

The term co-schedule, coined in [ST00], describes two threads running at the same time on two corresponding logical cores. Two questions in this section are: how can the performance

of a co-schedule be measured? And: is such a measurement comparable to measurements from previous intervals?

From a hardware perspective the question is: How high is the EU utilization of a co-schedule? A performance counter observing all executed micro-ops[1] could answer this question. However, if this measure accuratley reflects the performance degradation each thread suffers, due to sharing execution resources, needs further analysis.

To get the degradation percentage, knowledge about solo execution is necessary, which is either gathered offline beforehand, or by scheduling each thread alone for some time and extrapolating the measurement to the thread's lifetime. The former is excluded due to the scope of this thesis, the latter harms overall system performance and is imprecise, if the thread's behaviour changes heavily. Therefore, it is not possible to determine the precise degradation percentatge in this thesis.

Besides fine granular performance measure and relying on estimations, a third problem with rating HT performance is the length of a balancing interval and the possibility of running more than two threads during this interval on a pair of logical cores. If more than two threads are running during an interval, the load balancer can only rate the group performance, because it is impossible to deduce which of the $2^n - 1$ possible co-schedules[2] ran for which amount of time. Consequently, the group rating describes the performance of a physical core.

The same applies to any performance comparison between hyper-threads. As the performance of a single hyper-thread is influenced by its co-runner, it cannot be compared to logical cores of different physical cores.

All of this necessitates an abstraction from logical cores. This abstraction allows for the comparison of physical core performance and lets the load balancer act independently of the presence of HT. Furthermore, it provides a single point to change, if offline knowledge or hardware performance measurements become availble.

3.2 Isolation

Fiasco.OC provides isolation and also real-time guarantees. In the following, I show how a requirement-aware thread-to-core mapping of a load balancer can support these guarantees.

Security. As presented in section 2.2, even in virtualized environments the CPU cache is a valid side-channel to attack secret information. To reduce the channel's throughput, security-critical applications should share as little cache as possible with untrusted applications.

[1] UOPS_RETIRED.ALL

[2] The number of disjoint, non-empty partitions of a set of objects is computed with the *Stirling numbers of the second kind:* $S(n,k) : \quad n = |objects|,\ k = |partitions|$;
A special case exists for two partitions: $k = 2 : S(n,2) = 2^n - 1$.

In Haswell's inclusive cache hierarchy, the LLC contains the lines present in the core local L1 and L2 caches and is shared among all cores of a chip.

As shown in [Inc+15] and [Liu+15], the impact of placing security critical tasks on a seprate core is limited. Still, a security core increases the difficulty for PRIME+PROBE attacks. To eliminate this channel, no caches may be shared between the security-critical task and the attacker. Therefore, either a multi-socket system, Intel's Cache Allocation Technology can enforce this ([Int15a]), or similar cache rights management must be used.

The security requirements of a task are known at system design. Therefore, a way to provide this static information to the load balancing service is necessary. It is also preferable to define specific security groups to separate different security critical tasks or to create a group of related, equally trusted security tasks. Hence, the configuration scheme must allow to define groups and identify the same group in the configurations of different tasks.

This can be achieved by a per task configuration parameter identifying different groups by name.

Real-Time. A similar isolation guarantee is necessary for real-time tasks. Real-time tasks need predictable execution times. By using an exclusive core and cache, real-time tasks are less affected by the influence of best-effort workloads. Due to the inclusive cache design, evictions from the LLC require evicting the cache line in the core local caches as well. Hence, this approach limits the possible disturbance, but does not eliminate it. Also, the load balancer can assign several cores to a real-time task group, depending on its requirements.

As the requirements of real-time tasks are also known before run-time, a static configuration can be used. The configuration parameter must identify different groups by name and display the physical core requirements.

3.3 Communication

A less attended behaviour property of threads is communication. It is runtime-wise expensive to observe which threads are communicating with each other and how often. In the following I explore kernel and user-land approaches to acquire this information statically and dynamically.

Communication Patterns. I present two patterns of communication: compute-communicate and client-server.

Hofmeyr et al. ([HIB10]) assume a program running several threads, which compute a chunk of work and then communicate their results to each other. This cycle repeats until all work items are processed, whereas the time spent executing is the major part of the cycle and only

a small part is used for communication. Nevertheless, the overall program performance is best, if each thread reaches the communication synchronisation at the same time to spend as little time as possible waiting for others. [HIB10] notes this connection. This so called compute-communicate pattern is typical for distributed systems and is called bulk synchronous parallel programming (BSP) in high performance computing ([McC96]).

The other pattern is called client-server communication. To achieve the best possible performance, the communication latency, meaning the time difference between sending and receiving a message, must be minimal. The latency is minimal if both communication partners are on the same core. Additionally, time slice donation, where the server executes on the time slice of the client, is only possible in Fiasco.OC if both threads execute on the same core. Furthermore, the server will not execute unless it has a client request, so a server typically has long idle times. Therefore, it is preferable to execute both communication partners on the same core.

However, this conclusion is wrong, if the server has several clients from the same task. If the task runs several threads, then it most likely wants these threads to execute in parallel. Running all threads close to the server on the same core would contradict this intention and heavily impact their performance. In the worst case, the performance drops below single threaded performance, due to the overhead of context switching.

The same argumentation applies, if the server has a high number of clients from different tasks. Executing all clients on the server's core would be a waste of resources. Hence, the distribution of the task's threads to different cores takes priority over low communication latencies.

3.3.1 Kernel

Naturally, acquiring communication information at kernel level is easier than at user level, as more information context is available. However, access to kernel information from the user land must be pondered carefully. As the following two approaches rely on global object ids, which are a debugging feature, I consider them debugging features as well.

Communication Matrix. Communication partners as well as the rate of communication during a time frame, are of interest. It is important to reduce the latency of connections with a high message exchange rate. To model the communication relationships, each thread is assigned a global identifier and a matrix is created, where each field contains a counter, counting each communication event between the two corresponding global IDs. This matrix is symmetric, hence, to save memory, only the upper half is used and communication between two threads uses the counter in field (lowerID, higherID). The upper bound for the memory requirement is $½ * |threads|^2$. As not all threads will communicate with each other, usage of a sparse matrix will further lower the memory requirement.

Map. Another approach can be a thread local map, which contains the global identifier of the communication partner and a counter. Each communication attempt increases the counter by one. Furthermore, the used memory is directly accounted to the threads which need it. However, as this is a thread local solution, both communication partners will maintain their separate counters.

The upper bound for the memory requirement is $|threads|^2$, as each thread needs to maintain its own set of counters. As with the sparse matrix, only used counters are allocated to reduce the actual memory usage.

The user-land load balancer can use the system call interface to query and reset these counters. The response contains an array of all counters for this thread, to reduce the system call overhead.

3.3.2 User Land

At user level, information gathering is more complicated compared to kernel level. The strong isolation guarantees of Fiasco.OC make it infeasible to work with global identifiers across all threads. However, it is possible to ask the kernel if two capabilities point to the same object.

Cooperative Communication Capabilities. To communicate with a partner, a thread needs a capability in its own capability table, pointing to a channel to the receiver. A communication channel to another task is called an L4-IPC-gate. Together with a counter value, this capability of the L4-IPC-gate can be passed to the load balancer, which can then identify communication partners by comparing this received capability with the capabilities of the threads it manages. The load balancer can then build a relationship graph between threads and adapt the thread-to-core assignment.

This approach requires changes to the scheduler interface in L4Re and thread cooperation. However, it has the benefit that each thread can decide on its own if the amount of exchanged messages is high enough to benefit from increased locality. Unfortunately, this requires changes to each thread managed by the balancer and regular updates regarding the amount of messages exchanged. Together with the interface changes, this dynamic approach is deemed infeasible for this project.

Configuration Parameter. Less invasive, but static, is the usage of task specific configuration parameters. The two communication patterns I introduced at the beginning of this section yield two configuration parameters to represent the different behaviours. Together with an identifier they define thread groups. The load balancer then decides depending on the type of the thread group, if the threads should be placed close together on the same core, or distributed to as many cores as possible.

This approach requires careful system design, but no change of program source code or thread cooperation.

3.4 Energy Model

Energy consumption is important in modern systems, as battery powered devices become more popular. Measuring energy consumption is not part of this thesis. However, I will derive a model for energy consumption on x86 from work presented in section 2.2 and design an energy efficient load balancing strategy, based on this model.

A critical question for an energy model is: When is the least amount of energy consumed? In a race-to-idle scenario, where work is done fast and the cores sleep for a longer time, or if the work is done at lower frequencies for a longer time. In [LH11] Le Sueur and Heiser assume that quality of service is maintained, even at low frequencies. They show for their three applications that the energy efficiency is best for low frequencies, but depending on the application, at the cost of throughput. So it is highly application-specific to find the right frequency for the running application. The complexity increases, if more than one application is present in the system. Furthermore, a load balancer cannot decide on core frequencies, because its view on the system is not necessarily complete.

Le Sueur and Heiser note that turbo boost should be avoided at all cost, as the performance gain is little compared to the increased energy consumption. The hardware does not use turbo boost as long as all cores are loaded, as the energy budget does not allow it. Also, it is possible to disable the feature completely.

Another relevant observation in [LH11] is, that the usage of deep sleep states and the related restart costs only slightly increase the system load. If a core goes into a deep sleep state, the system load is low anyway, so the additional wake-up load poses no problem.

[Ime+15] states that different models are necessary for different platforms. A mobile Haswell generation processor consumes the least amount of energy if the race-to-idle scheme is used, whereas an embedded Exynos5 generation CPU is designed for constant low frequency operation. The target hardware for this work is a Haswell generation processor, so the race-to-idle model seems promising.

The difference in energy consumption between low frequency and normal frequency, however, is little compared to a deep sleep state, due to the already high base energy consumption. So following [Ime+15], it is best to increase sleep time and use the normal frequency to execute the arising work quickly.

The model parameters for best energy efficiency are: No turbo boost, either execute at normal frequency or sleep, and use as many cores as possible, to reduce cache misses and consequently execution time.

A reduction in run-time leads to larger sleep times or more throughput, which is the goal of a load balancer all along. Therefore, an algorithm maximizing throughput also increases energy efficiency on the x86 architecture.

3.5 Load Balancer

I now unit all the techniques I introduced in the previous sections of this chapter into the central load balancing component. In this section I describe the components the load balancer is built of.

Load Balancer & Task Proxies. To achieve good load balancing performance, the load balancer needs information about the underlying hardware, as well as the total amount of work in the system. It is also possible to build hierarchical load balancing, where only a partial view on the hardware is available and only a subset of tasks run on this specific load balancer. However, while leading to isolation between different subhierarchies, the overall performance can suffer, as the system is effectively partitioned. A surplus of computing resources in one partition is unusable by another partition leading to wasted resources. The size of the partition is only adaptable at core granularity, so meeting the precise resource needs may not be possible, leading to overprovisioning and, consequently, resource waste.

Isolation between parts of the system can also be provided by a central load balancing component. A central load balancer as well as a hierarchical approach are part of the chain of trust of each balanced application, so both approaches are equally trustworthy. However, the central component can respond dynamically to changes in the workloads and shift the partition borders as needed. Also, if tasks with security properties, as discussed in section 3.2, enter or leave the system, the dynamic approach can repurpose a core. Redistribution of cores between hierarchical load balancers would require additional reasoning which load balancer the free core is assigned to, or which load balancer must release a core. The load balancers must communicate their reasoning either with each other, or with a central instance further up the hierarchy. The former weakens the isolation and increases overhead, the latter means more logic in a central instance. Due to the easier implementation of a central load balancing component, I choose to implement the centralized service.

Internally, the load balancer uses task-specific proxy scheduler to store the task's configuration information. Each task can have isolation (3.2) and communication properties (3.3). The

task's proxy scheduler stores the configuration and provides it to the central load balancer, as depicted in figure 3.1.

A factory creates the central load balancer and processes the configuration parameters for each client task. The client's proxy scheduler stores the configuration structure and provides it to the central load balancer. The configuration structure comprises the priority range and a list of groups derived from the parameters.

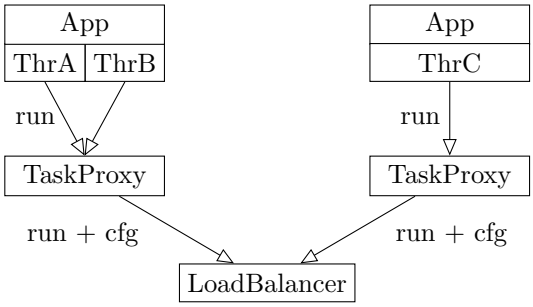


Figure 3.1:
Load Balancer with one Task Proxy per client task. The Task Proxy adds the task's configuration to the `run_thread` call.

Thread Management. All threads known to the load balancer are managed by the thread management. It creates an internal administration class for each thread and identifies a thread via its capability. The load balancer evaluates the configuration passed by the task proxy and if isolation or communication groups are set, the thread is assigned to these groups. The management provides a consistent view on all managed threads for other components and is also in charge of cleanup.

CPU Topology & Accounting. Besides the management of threads, the load balancer must also know the topology of the underlying hardware. The topology scanner analyses the topology via CPUID and stores it in a hierarchical data structure. A package describes all physical cores of one socket. If HT is present, each physical core representation contains data structures for two logical cores.

Each physical core is associated with an accounting object. Core accounting stores all assigned threads and computes the accumulated load on its assigned core. Additionally, it provides information on which thread to migrate in case of an imbalance. The accounting object connects topology, physical cores, and threads, therefore, it is the place for the SMT abstraction introduced in 3.1. Each thread assigned to the physical core is distributed to its hyper-threads by an algorithm, e.g. round robin, LLC weight, IPC, or load. If no HT is available, no further assignment is necessary. The availability of HT is dynamically deduced from the topology enumeration results.

Information Gathering & Balancing. The logic of the load balancer is partitioned into four consecutive modules: measurement, prediction, decision, and enforcement. Each balancing interval, they run once and adapt the thread-to-core assignment to the current circumstances. Figure 3.2 depicts this Measure-Predict-Decide-Enforce (MPDE) cycle.

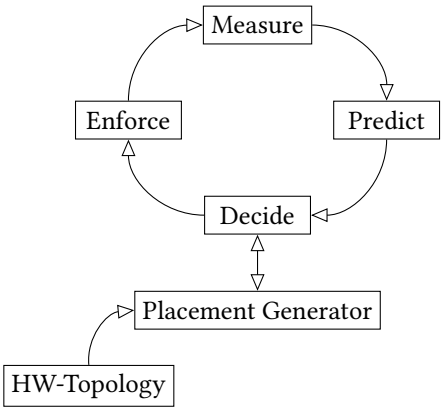


Figure 3.2:
Sequential succession of the measure-predict-decide-enforce cycle inside the load balancer. This cycle executes every balancing interval to read performance data and to decide on migrations.

First, the information about the current situation is updated by gathering performance counter and execution time values of the last interval for each thread. Based on these measurements, the prediction module generates a forecast for each thread's resource needs. This forecast forms the input for the decision module. Together with the topology information and the accounting values for each core from last interval, the decision module checks if the system is still balanced. If not, selected threads must be migrated to establish balance again. After deciding which threads must be migrated, the enforce module applies the migrations by modifying each thread's affinity and informing the kernel.

These steps repeat every interval. A high interval duration reduces the load balancing overhead, but also leaves the system in a bad state for a longer time, when thread behaviour changes or threads enter and leave the system. A short interval duration increases the measurement overhead and may increase the amount of migrations. The ideal interval size increases system performance to outweigh the interval's own run-time overhead.

The benefit of dividing the MPDE cycle into four separate components lies in its flexibility. If other information sources should be used, the measurement component is the place to add them. If different performance prediction algorithms should be tested, only the prediction component changes. The decision component allows to easily test balancing and placement algorithms and also incorporating multi socket systems is neatly encapsulated in it.

Of course, if information sources arise, changes down the line are necessary, but this approach provides a concise description of the different components. Also, extending the load balancer to heterogeneous architectures is assumed to work well, as the design is similar to the approach in [Sar+15].

3.6 Load and Balance

What is load? And depending on the answer to this question: What is balance?

As I describe in 2.3, CFS defines a core's load as the amount of execution time used by threads. However, a core's load can also be defined as the cache and memory usage, the sum of IPC, or simply as the number of assigned threads. It is also imaginable to use a mix of the above as load measure.

The answer to the question: "What is balance?" depends on the definition of load. Assuming the CFS definition, a domain is balanced if the groups within the domain carry an even amount of load. If this assumption holds for each level in the domain hierarchy, the system is in balance. However, load only represents consumed computation time, thus, it balances time and disregards space completely.

The opposite approach measures only LLC misses per cycle (MPC) and ignores computation resources. So the load of a domain is the sum of MPC of all assigned threads. A system is in balance, if each group within a domain has an equal number of MPC, or in a different formulation: if no thread-to-core assignment reduces the imbalances between cores. However, this metric suffers because of its disregard for required execution time. While memory intense threads may perform better, computation intense threads suffer due to migration and possibly thread count imbalances between cores. For example, if one thread produces a high number of MPC it is assigned a core exclusively, to meet its cache requirements. If four other threads are computation intense and the sum of their MPC is still lower than the high MPC threads, they will be sharing one core. This system is in balance regarding MPC, but regarding execution resource it is perceived as heavily imbalanced.

A third metric is IPC, which similar to load is a temporal balancing metric. It shows, how much instructions a thread was able to perform during its run-time. Similar to MPC, this number is different for each thread. CPU bound workloads tend to have a high number of IPC, whereas memory bound workloads tend to have a much lower number, due to cache misses and waiting for memory. Also, instructions are not equally costly, which leads Snavely and Tullsen ([ST00]) to their weighted instruction measure. However, at run-time it is not possible to pinpoint the weight of the current instruction.

Balancing only by IPC will lead to a similarly unfair distribution of threads to cores as balancing by MPC. The disregard for cache usage will likely decrease the number of instructions a cache sensitive thread is able to execute, as more cache misses are generated.

The number of threads assigned to a core also impacts the progress of other assigned threads. However, only if the newly assigned thread is really executing and not a service, which mostly idles and executes only on request. Thread count ignores load, IPC, and MPC and is therefore not of much use.

A mix of execution time, IPC, and MPC looks promising. The main drawback of execution time is its obliviousness of cache usage. Extending the execution time metric with MPC knowledge should alleviate this. A second approach can combine IPC and MPC metrics: I assume IPC and MPC to be two orthogonal measures, hence I expect running a high MPC thread on the same core as a high IPC thread to perform well.

After discussing load and balance, it is also necessary to decide on a threshold, before actively migrating threads. The cost of a migration must not necessarily be paid to have a perfectly balanced system. Slight imbalances decrease the performance compared to a perfectly balanced system, but as long as this decrease is less than the cost of a migration, it is preferable to tolerate the imbalance.

Cache-heavy threads, determined by last-level cache misses per cycle, can have two reasons for a high miss count: first, they reuse little memory and use new memory in a not prefetchable manner, otherwise the prefetcher would not miss; second, the cache working set is large and parts of the working set were evicted during the run-time of other programs. Migrating the former kind does not hurt the program much, migrating the latter kind, however, is costly as the complete cache working set must be reloaded into the new core's cache.

While it is generally a bad idea to migrate high MPC threads, a clustering of high MPC threads is worse and must trigger a migration of a one of the high MPC threads.

In conclusion, balancing should take place in time and in space. While balancing in space takes priority over balancing in time, temporal balancing evens out imbalances originating from spatial balancing. The spatial balancing should act, if high imbalances regarding accumulated MPC per core arise, or if the difference of MPC-heavy threads per core rises above one or two high MPC threads per core.

Regarding temporal imbalances, the implementation questions are: is there a thread that would even out imbalances if migrated? And: At which load differences does the load balancer start searching for such a thread?

3.7 Summary

During this chapter I discussed the difficulties with determining the performance of corresponding hyper-thread cores without offline knowledge about the running threads. The HT abstraction achieves two things: the load balancer compares the performance of physically equal units of hardware, and it becomes independent of the actual presence of HT.

The load balancing service provides isolation for applications with real-time or security requirements. Also, I presented different approaches to gain knowledge about communication relationships within a task or between a group of tasks.

The review of energy models led to the conclusion, that on x86 processors a race-to-idle scheme without usage of turbo-boost features is the most promising model. A reduction in computation time leads to larger idle periods for the different cores, which in turn leads to more time spent in deep sleep states, where energy usage is minimal.

Subsequently, I presented my design of the central load balancing service and its modules. Measurement and prediction of thread behaviour provides the knowledge to decide on balancing options and to enforce this decision.

The chapter finished with a discussion about different definitions for load and depending on the character of the load, which system states are considered to be in balance.

The following chapter presents changes to the Fiasco.OC kernel to provide access to hardware performance counters; how a system designer informs the load balancer about isolation and communication groups; and discusses different algorithms to decide on thread placement.

4 Implementation

In this chapter, I explain implementation details of the load balancer. I start with the description of the kernel implementation of the performance counter, together with the user-land interface to query said counters. Then, I describe the task configuration and processing of the configuration by the load balancer. I follow up with a description of internal data management during a balancing cycle. Subsequently, I present load balancing algorithms.

4.1 Kernel Performance Counter

Intel provides hardware performance counters for various CPU architectures. Their intention is to enable software developers to observe the behaviour of their software and determine bottlenecks and hotspots.

I presented the performance monitoring capabilities of the targeted Haswell architecture in section 2.5.

The current Fiasco.OC implementation for architectural performance monitoring is extended to use the fixed-function counters for unhalted core cycles and instructions retired. Additionally, a general-purpose counter for LLC misses is used, leaving three general-purpose counters per logical core for future use.

Configuration & Start. Each general-purpose performance counter consists of a counter register and a configuration register. To specify the event the counter should observe, the `UMask` and `Event Select` fields in the configuration register must be filled. Additionally, user and operating system mode bits must be set, to count events happening in privileged and unprivileged mode. With the edge detect bit set, every new occurrence of the selected event is counted. To start counting, two enable bits must be set. The first in the counter's configuration register and the second in the global control register. If one of these two bits is not set, the counter is not enabled.

Fixed-function counters are a bit different. They consist of one counter register and one control register shared among all fixed-function counters, to enable and disable each one. Like general-purpose counters, the fixed-function counters must also be enabled in the global control register.

Fiasco.OC already contains the boot parameter `-loadcnt` enabling the unhalted core cycle counter on each core. Although a fixed-function counter is available for this event, the implementation still used a general-purpose counter. Using this flag, each core's performance counters are configured and enabled to count unhalted core cycles, instructions retired, and LLC misses.

Per Thread Accounting. The `-loadcnt` flag only enables the hardware counters. The accounting must be enabled for each thread separately, by calling `l4_thread_perf_reset`, which also resets the counter value to zero. Only then, Fiasco.OC adds the counter values on each context switch to the thread's account and resets the hardware counter to zero for the switch-to thread.

I extend the `L4::Thread` interface to access the thread's performance account. `perf_read` provides the interface to retrieve a thread's current account. A `perf_counter_read_t` struct passed by reference is filled with the values provided by the kernel. Listing 4.1 shows the user-land implementation of this interface.

```
enum L4_thread_ops {
  ...
  L4_THREAD_PERF_READ_OP        = 10UL,
  L4_THREAD_PERF_RESET_OP       = 11UL,
  ...
}

typedef struct perf_counter_read_t {
  l4_uint64_t core_cycle_cnt;
  l4_uint64_t instr_retired_cnt;
  l4_uint64_t llc_miss_cnt;
} perf_counter_read_t;

L4_INLINE l4_msgtag_t
l4_thread_perf_read(l4_cap_idx_t thread,
                    perf_counter_read_t *cntr) L4_NOTHROW;

L4_INLINE l4_msgtag_t
l4_thread_perf_reset(l4_cap_idx_t thread) L4_NOTHROW;
```

Listing 4.1: User land interface for retrieving and resetting a thread's performance counters; UTCB-pointer left out for brevity.

4.2 Task Configuration

In sections 3.2 and 3.3, I discussed task properties relevant to a load balancer. The result of these discussions was to provide the load balancer with a static configuration of a task's properties. In the following section I show how the configuration is used and implemented.

Configuration Parameters. Listing 4.2 shows the implemented parameters: `sec` and `rt` define security and real-time groups; `distr` and `clsvr` configure BSP and client-server communication groups. The `_C` or `_S` flag marks a task as client or server.

```
min_prio  = <number>
max_prio  = <number>
sec       = <name>
rt        = <name>
distr     = <name>
clsvr     = <name>_{C|S}
```

Listing 4.2: Implemented set of configuration parameters for priority range, isolation, and communication groups.

The system designer must not define orthogonal parameters for a single task. Either a task is in a security or in a real-time group, never in both. The same goes for `clsvr_C` and `clsvr_S`: A task cannot be client and server within the same group. Also, `distr` and `clsvr` may not be combined, as the `distr` property is assumed if a client task runs more than one thread. Additionally, the system designer may not define more real-time and security groups than physical cores available. As mentioned in 3.2, a real-time group could have more than one core assigned, but this is currently not implemented. In the future the name could be followed by a core count, to state the number of cores necessary.

To configure the task a create call to the factory service of the load balancer must be issued, containing the scheduler protocol as first parameter and then a comma separated list of strings stating the configuration parameters. The factory will then return a capability on a channel to its proxy scheduler encapsulating the task configuration.

The capability received from the above create call replaces the from the parent inherited scheduler in the task's environment. As the scheduler interface has not changed, the task accesses the scheduler via `L4Re::Env::env()->scheduler()`. Listing 4.3 displays a complete task creation in the Ned configuration script.

```
L4.default_loader:start(
  {
    caps = {},
    scheduler = tmFactory:create(
      L4.Proto.Scheduler,
      "min_prio = 0", "max_prio = 5", "distr = A")
  },
  "rom/app_matrixmul"
)
```

Listing 4.3: Task creation with create call to the load balancer factory and replacement of the default scheduler in the Ned startup script.

Therefore, it is only necessary to change the the application startup configuration parameters, which are encapsulated in the Ned startup script, shown in listing 4.4. The load balancing service is completely transparent to the application.

```
local L4 = require("L4")
local aux = require "rom/aux_module"

local ld = L4.default_loader
local threadMapperFactory = ld:new_channel()

ld:start({ caps = { thrapFac = threadMapperFactory:svr() },
           log = { "server", "blue" } },
         "rom/thrap")

local ITERATIONS = 100

local durations = aux.restart( ITERATIONS ,
    "rom/app_matrixmul_parallel",
    {},
    {"mmul", "green"},
    threadMapperFactory,
    {L4.Proto.Scheduler, "min_prio = 0", "max_prio = 10"}
  )

aux.printTableBench(durations)
```

Listing 4.4: Example of the a benchmark startup using the load balancer as scheduler.

Configuration Groups. The configuration arguments stated in the startup script are parsed by the factory service and stored in a `config_t` type (listing 4.5). The parser strips all white spaces from the configuration string and splits it in two parts: before and after the equality sign. The front part evaluates to priority or group type, whereas the rear part evaluates to either an integer or string. If the group type is a client-server type, the last character is matched against `C` or `S` to recognize client and server tasks.

Note, this configuration is per task, hence all threads of this task possess the same configuration. A configuration group must not necessarily span across several tasks. The `distr` parameter, for example, can group all threads of one task and ensures that all threads of this task will be assigned to the maximal number of available cores.

```
struct threadMapperConfig {
  struct GroupInfo {
    enum class GroupType {
      RT, SEC, CL_SVR, DISTR,
    };
    enum class ClSvrType {
      NONE, CLIENT, SERVER,
    };
    const string name;
    const GroupType group;
    const ClSvrType clsvr;

    GroupInfo(string n, GroupType g, ClSvrType c = ClSvrType::NONE)
        : name(n), group(g), clsvr(c)
    {}
  };
  using GroupInfo = struct GroupInfo;

  l4_umword_t minPrio;
  l4_umword_t maxPrio;
  std::list<GroupInfo> groups;
};
using config_t = struct threadMapperConfig;
```

Listing 4.5: Implementation of the configuration type stored by the task proxy.

4.3 Internal Data Management

If a thread calls `scheduler()->run_thread(cap, sched_params)` passing a capability and its scheduling parameters, the proxy scheduler receives the call. After adding the task's configuration, the proxy forwards the run call to the central load balancer by invoking `run_thread(cap, sched_params, config)`. The load balancer then asks the thread management to look up the capability in its list of known threads. If the thread is unknown, a new internal thread object of type `Thread_t` is created, encapsulating the capability, the scheduling parameters, and the configuration. Besides these three values, `Thread_t` stores previous and current performance values and execution times.

Next, the group configuration is evaluated and the `Thread_t` instance is assigned to all configured groups.

Besides creating and searching, the thread management also validates the presence of threads and in the case of deletion, it removes the corresponding `Thread_t` instance from all groups, management lists, and deletes empty groups, before the object is destroyed.

MPDE Cycle. The interval cycle design introduced in section 3.5 is implemented as follows: Measurements and prediction are stored in the thread's `Thread_t` instance. The decision component evaluates the balance of the system and selects threads to migrate. The new scheduling parameters are also stored in the thread's `Thread_t` instance. If the scheduling parameters changed, the enforce component informs the kernel. `Thread_t` is the central structure connecting all these components.

Threads in groups have the benefit to be placed separately, maintaining their group properties. Also, when the system is unbalanced, threads without group associations are considered first for migration. Isolation groups are not considered here, as their core assignment is static. Then the threads of each `distr` group are distributed to all cores, before `clsvr` type groups are placed. The client-server groups are placed on one core. If a task provides several threads to the group, its threads are evenly distributed to each core.

Afterwards, the load balancer assigns the threads without group affiliation. First, the cache-heavy threads are distributed to each cache, depending on MPC; they are balanced in space. Second, the remaining threads balance the system in time. The load balancer tries to even out all remaining imbalances regarding computation time between the cores. The following section describes this in more detail.

4.4 Algorithms

The load balancer uses different algorithms for SMT placement, assignment of groups, and balancing of threads without group affiliation. I present algorithms for these three placements and also a simple behaviour-unaware load balancing algorithm. The simple load distribution algorithm serves as a baseline to demonstrate the benefits of behaviour analysis.

SMT distribution. After the physical core assignment is done, each physical core internally distributes its set of assigned threads to the hyper-threads. Different options arise: simple round robin, by MPC, by IPC, or by load.

Another possibility consists of one core getting the high MPC threads and the other one the threads with high IPC. I assume this leads to a good symbiosis, as the high IPC threads compute, while the MPC-heavy threads wait for memory.

Superiority of one of these options above the others must be empirically evaluated. The default balancing uses CPU load as metric.

Two-Dimensional Balance. What is balance from a implementation point of view? This question takes the most effort to answer, because it depends on several factors: isolation groups reduce the core count; communication groups are excluded from load balancing; and only threads without further affiliation are considered for migration. So balancing works on top of the placement of communication groups, which is possibly already imbalanced. The load balancer must even out these imbalances and be aware of cache usage of these threads as well. Also, if only parts of the system are managed by the load balancing service, the base load needs to be considered during placement. All this leads to a difficult environment for the load balancer and complicates the test for the overall system balance.

The core accounting needs to keep track of unaccounted load, accounted load by not migratable threads, and accounted load by migratible threads. MPC measurements are only accessible for managed threads, meaning threads within groups and threads without group.

As mentioned in section 3.6, balance is two-dimensional. Spatial balance takes priority over temporal balance, to allow better cache usage. But at the end, temporal balance is equally important.

A simple spatial balance metric is the following: Sort the migratable threads by MPC and assign the top four to one core each.

To adapt to preexisting imbalances regarding MPC, the number of chosen threads can vary; for example if the difference between the core with the highest MPC and the core with the second highest MPC is larger than the MPC-heaviest thread, assigning another thread to it makes no sense.

Temporal balancing is different, as it has to assign all remaining threads and consider the absolute core load, not only the load of the managed threads. Sorting the threads by load, and assigning them one by one to the core with the least load, leads to a balanced system.

When imbalances in time are detected, the temporal balancer computes the difference in consumed CPU time and searches for a thread to migrate. Ideally, the thread generates load that accounts for half of the difference. If no such thread is found, the thread generating the next higher amount of load is migrated.

MPC-IPC-Placement. This algorithm creates two copies of the list of all threads and sorts one copy by MPC and one by IPC. The first item of the MPC-list is deleted from the IPC-list. Then, the algorithm deletes the first item of the IPC-list from the MPC-list. Both lists are processed item by item, until the two lists are disjoint.

As long as both lists contain more threads than cores available, each core is assigned one thread from each list. If the number of threads left in both lists is less than the number of available cores, the algorithm assigns these remaining thread to the core with the least MPC and IPC, respectively, to keep the number of threads per core even.

The performance of this approach must be evaluated. However, this approach does not account for execution time, therefore a possible improvement is to assign the remaining threads by execution time instead of IPC and MPC.

Group placements. Configured groups are treated with a higher priority, because their dependencies are known to the load balancer and must be treated accordingly. The group type `distr` suggests blocking synchronization among all group members. Thus, they are distributed to all cores to provide equal progress opportunities to each member.

`Clsvr` type groups are handled in two ways: First, it is checked which group members belong to the same tasks. Of each task one thread is assigned to the same core as the server thread, the rest is distributed to the other cores.

`Sec` and `rt` type groups get their private physical core, hence all threads of these groups are assigned to the same core. However, a security critical application might not need to run all the time, e.g. email decryption, thus if the application is stopped, the exclusive core is reassigned to the general load balancing. If the application is later started again, the load balancer provides an exclusive core.

Simple Load Distribution. The algorithm assigns incoming threads to the core with the least load. If several cores have the same load, e.g. in an overload scenario, the algorithm assigns the thread to one of the cores with the least threads.

A core's load is determined by the amount of idle time since the last placement. When threads leave, the system becomes unbalanced, but the algorithm does not take extra balancing actions.

To prove that behaviour analysis brings any benefits, it has to beat this simple algorithm.

Missing Logic for Shared Memory. Tasks running several threads which use shared memory to communicate will have a worse performance using these algorithms. No communication parameter tells the load balancer that these tasks should be grouped together. Consider two threads with shared memory on two separated cores. The only connection is the LLC. The cache lines holding the shared memory will bounce back and forth between the two cores, as one writes and invalidates the cache of the other. This means additional L1 and L2 cache-miss costs, each time they communicate.

If the number of threads is equal to the number of cores, the cache line will bounce between four different L1 and L2 caches, impairing the overall performance of the threads.

An additional shared memory configuration parameter is necessary to help the load balancer diminish this issue. Then two threads could be placed, such that they run co-scheduled on the same caches.

4.5 Summary

Implementation characteristics of the prototype load balancing service were the main topic of this chapter. The kernel provides per thread accounting on a set of predetermined hardware performance counters and defines an interface to access a thread's account from user land.

These measurements are the main information source for the load balancer. The other information source is the static task configuration defining isolation and communication groups. The system designer provides these information via the task startup script to the load balancer.

This chapter finished with a discussion about different algorithms for the SMT abstraction, group placement, and balancing in space and time. Also, I presented a simple load distribution algorithm to provide a baseline for evaluation.

The next chapter evaluates the prototype's performance by comparing it to the simple load distribution algorithm and Linux's CFS.

5 Evaluation

Numerous questions regarding different aspects of load balancing arise throughout this thesis. What is the performance impact at SMT-level of good balancing? How does the prototype perform under light, normal, and heavy load compared to Linux's CFS? Load balancing is not only about average performance, it is also about consistent run-time behaviour of the whole system throughout several repetitions. How do the balancing algorithms perform in this regard? How beneficial is the knowledge of communication relationships? How does a preferential treatment of those impact the performance of other applications in the system?

These are the questions I delve deeper into in this chapter. However, due to time constraints, I am not able to measure, analyse, and answer all of them in detail.

5.1 Benchmark and Measurement Setup

In the following, I describe the benchmark applications and the system environment the benchmarks run in.

Benchmarks. I use four single threaded applications from the *SPEC CPU2006* benchmarking suite[1]: namely, 403.gcc, 416.gamess, 429.mcf, and 470.lbm. The benchmark set consists of two compute-bound and two memory-bound benchmarks, with one integer and one floating point benchmark within each group.

416.gamess is a floating point benchmark written in Fortran executing a quantum chemistry simulation. It is a compute-bound benchmark and generates close to zero MPC. The 403.gcc integer benchmark compiles a fixed amount of source code. It has nine different input workloads, from which I selected the 166.i workload. The benchmark is also compute-bound and generates a low number of MPC. 470.lbm simulates fluid dynamics and stresses the floating point units. It generates a high amount of MPC, therefore it is memory-bound. 429.mcf generates the highest number of MPC. It computes a combinatorial optimization using the integer units.

Algorithms. I will measure the performance of four different algorithms I introduced in section 4.4. The simple load distribution (SLD) algorithm distributes each new thread to the core with the largest amount of idle time. It does no further migrations.

[1] `http://spec.org/cpu2006/`

The space-time-balance (STB) algorithm observes temporal and spatial thread behaviour. It distributes the threads with the highest memory usage to different caches and balances the rest depending on their consumed execution time during the last interval.

The third algorithm is a small adaption of the space-time-balance algorithm. It omits the spatial balancing and solely performs temporal balancing (TB).

The fourth and last algorithm differs from the above, as it runs in conjunction with the kernel-level scheduler; it is Linux's CFS.

Time Measurement. For each application I measure the time it takes to start the application, execute all threads to completion, and exit. I repeat the measurement at least 100 times. When several benchmarks run concurrently, the slowest benchmark needs more time to reach 100 measurements. To keep the system load constant, fast benchmarks – 403.gcc and 416.gamess – provide more measurements than slow benchmarks – 429.mcf and 470.lbm.

On L4Re, I use the Ned startup script to measure and output the times for each application. On Linux, I use a self-written ruby script to measure the durations and to store them in one file per application.

Furthermore, I do not use the time stamp counter (TSC), as this counter depends on the core the time stamp is taken on. Instead I use `clock_gettime(CLOCK_MONOTONIC)` on Linux and `l4_kip_clock(l4re_kip())` on L4Re.

64bit Issue. Running the benchmarks on 64bit raises an issue in the memory subsystem of L4Re. Memory allocation and deallocation increases the run-time of 403.gcc by several seconds for each repetition. The load balancing algorithm is not the cause for this behaviour, as this issue also presents itself if no load balancing is done. The task runs to completion and then restarts, so I expect each allocated page to be freed after completion. The L4Re internal memory management should now merge free chunks of memory. However, the size of the list of free memory areas increases constantly and shows allocated pages in between, preventing compaction. I suspect the time spent iterating through the growing free list to search for possible compactions or chunk sizes fitting a memory request to be the cause for the increase in run-time.

This issue does not visibly affect the run-time of 416.gamess, but the other memory-bound benchmarks are affected as well. 429.mcf shows slight but constant increase with each run. 429.mcf allocates all memory at the beginning of the benchmark, so the memory subsystem is not used constantly, as is the case for 403.gcc.

When 416.gamess, 429.mcf, and 470.lbm run together on L4Re, the run-times increase occasionally by a factor of 2.5 to 4. This increase cannot solely be explained by bad load balancing decisions. Also, the average run-time for the first 50 iterations of 470.lbm is 60 seconds lower

than the average run-time of the second 50 iterations. Minimum, maximum and median run-time show a similar difference.

Providing a different memory allocator to each task via the Ned script did not eliminate this problem. As this issue did not present itself for 32bit, all benchmarks are executed in a 32bit environment. This issue will be addressed by the L4Re developers.

5.2 Measurements

I measure three different constellations of the SPEC applications: solo, load, and heavy load. The solo execution time of each application serves as baseline to compare the performance of different algorithms in a low load situation, and to enable the computation of the slowdown when run in parallel with other applications. I use the terms execution time and run-time synonymously and they describe the duration in wall clock time units between two measurements. I described the measurement process in the previous section.

As a means to visualize the run-time measurements, I chose box plots. However, due to the nature of the box plots, data points marked as outliers need to be taken with a grain of salt. Box plots have six components: median, lower and upper quartile, lower and upper whisker, and outliers. The distance between the lower and upper quartile values is the *inter quartile distance* or IQD. Depending on the IQD, the lower and upper whiskers are computed as follows:

$$\text{lower whisker} = \text{lower quartile} - 1.5 * \text{IQD}$$
$$\text{upper whisker} = \text{upper quartile} + 1.5 * \text{IQD}$$

The actual value of the lower whisker is the value of the next data point that is higher than the computed value. For the upper whisker it is the next lower-value data point. Box plots consider every value beyond the value of the lower or upper whisker as outlier.

The issue arises, if the IQD is low and therefore, the whisker values are close to their respective quartile value. Then a lot of data points can be marked as outliers. While they deviate from the bulk of run-time values, the actual distance is not necessarily high. Consequently, a high number of marked outliers is not necessarily a sign of high deviation.

Cost of Balancing. Balancing can be expensive run-time wise. If it migrates threads with a large cache working set to another physical core, the threads need to reload their working set into the core local caches. Furthermore, the MPDE cycle needs processor time. Still, the housekeeping during the cycle can be expensive. Because threads can leave at any time, the load balancer must check their presence multiple times and act on failing system calls. I believe optimizations to the source code can reduce the time, but currently, the MPDE cycle takes around 55µs when managing one thread, 170µs when managing ten threads. However, I

observed that this cycle can take up to two orders of magnitude longer due to housekeeping issues. The cycle duration depends on the number of threads the load balancer manages and not on the length of a balancing interval. A shorter balancing interval, however, raises the total overhead of the load balancer, as the MPDE cycle is executed more often. The overhead analysis raised questions about the implementation. In the worst-case, an MPDE cycle can take up to 1.5s. As the measurement component gathers each thread's performance data, it has to send a message to the core, that currently runs this thread. This leads to several cross-core messages and the answer is also delayed, as the request has the same priority as the running threads. To improve this, the load balancer must employ one measurement thread per core with a higher priority.

In the following, I analyse, how many cache misses must occur, to be the reason for a one second run-time increase of an application, when running alone on the load balancer. Intel documentation ([Int09]) states 60ns for an LLC miss, independent measurements[2] suggest 70ns, which accounts for a difference of 3 to 4 cycles on a 3GHz CPU compared to the Intel statement.

The maximum amount of misses per second is $10^9/70 \approx 14285714$ misses. During one interval of 100ms $10^8/70 \approx 1428571$ or 10ms $10^7/70 \approx 142857$ misses can be generated.

The memory-heavy Mcf application runs for 90 seconds. When its run-time is increased by one second due to bad balancing decisions and subsequent cache misses, it suffers on average an additional 14285714/90 ≈ 158730 cache misses per second during its run-time. Relative to an interval length of 100ms that means 15873 misses per interval. The cache size of the core local L1 and L2 caches is 64 + 256 = 320KB or 5120 cache lines à 64Byte. Producing 15873 misses during an 100ms interval would mean to flush the whole cache 3.1 times. One cache miss takes 70ns, hence flushing the whole cache 3.1 times takes $5120 * 3.1 * 70ns = 1111040$ns or 1.11104ms. So, if Mcf suffers a slowdown of one second it has to spend 1.1ms each balancing interval on additional cache misses.

However, forcing a core switch every 100ms might cause losing a whole cache set once, but not thrice. This discussion is also simplified, as I discuss last level cache misses and only account for core local caches. When a core switch occurs, it is likely that the data can be fetched from the LLC as it is an inclusive cache. A successful LLC access takes approximately 13ns. Assuming reuse of all 5120 cache lines, fetching them takes 66560ns or 0.06656ms. That leaves around 1.05ms per balancing interval to account for.

This concludes that additional cache misses due to load balancing decisions cannot be the sole reason for run-time deviation as observed in figure 5.1.

Solo Execution. The solo run-times for each benchmark provide insight into the minimal execution time and the program behaviour throughout different runs. However, the benchmark

[2] `http://www.7-cpu.com/cpu/Haswell.html`

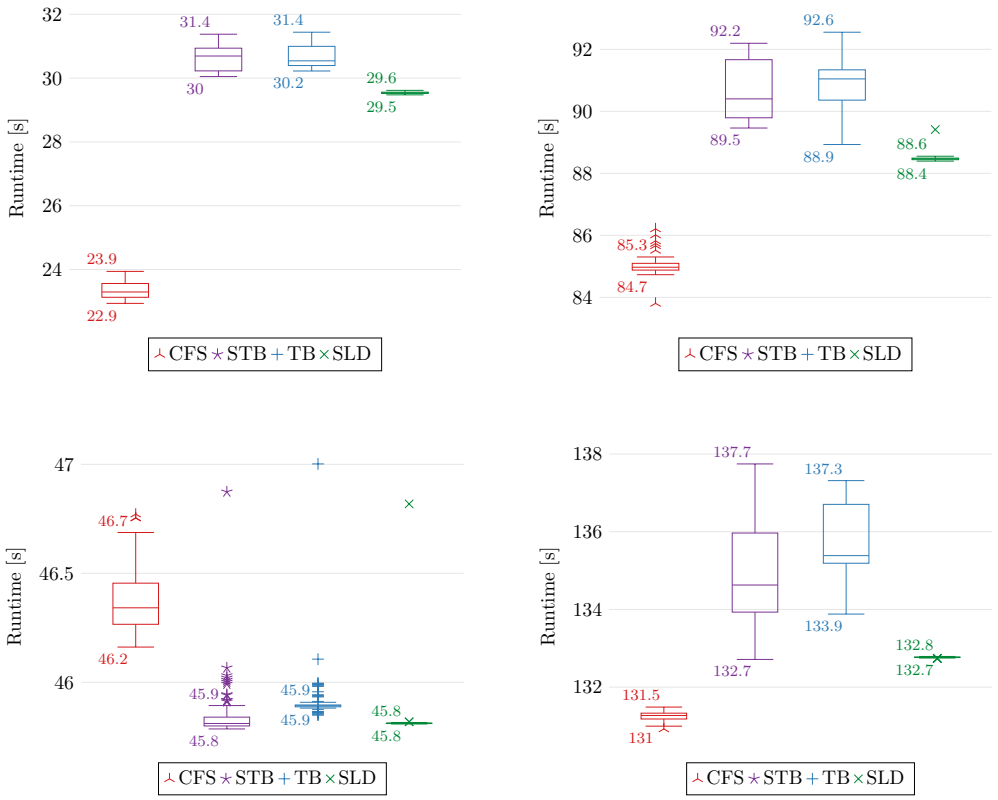


Figure 5.1: Solo run-times visualized using one box plot for each balancing algorithm; STB and TB use a 100ms balancing interval. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm.

is not the program under test, the balancing algorithm is. SLD achieves the minimal possible run-times, because it does not migrate any running application and produces no additional overhead through measurement and balancing action. The green box plots in figure 5.1 show that nearly all SLD run-times lie within a tenth of a second of each other. The very deterministic run-time suggests that SLD schedules the benchmark on the same core each run. Also, compared to the violet and blue plots of STB and TB, the SLD run-times are either as good as the minimal run-time of the other two or better.

CFS on the other hand is significantly faster, except for Gamess. The gap for the memory-bound benchmarks is smaller than the gap for the compute-bound Gcc. CFS dynamically adjusts the time slices of the running program to reduce the scheduling overhead. Furthermore, the amount of time, money, and effort the Linux community put into scheduling is reflected in these numbers.

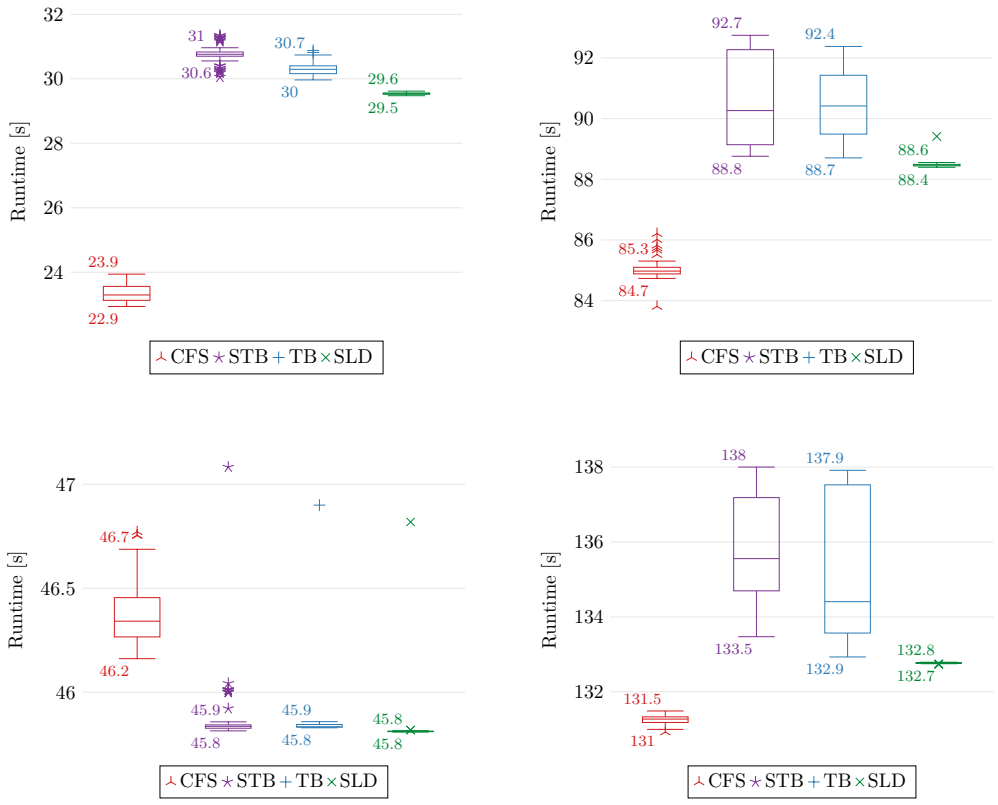


Figure 5.2: Solo run-times visualized using one box plot for each balancing algorithm; STB and TB use a 10ms balancing interval. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm.

The results of STB and TB display equally fast run-times for the compute-bound benchmarks. However, the run-times of the memory-bound benchmarks raise questions about the amount of migrations and the overhead of the load balancer. Between figure 5.1 and figure 5.2 lies a tenfold increase in overhead, as the interval duration changes from 100ms to 10ms. In case of Mcf, STB displays better run-times for best-case, worst-case, and median, but worse run-times at all three points for Lbm. TB's results for Mcf improved as well. Whereas Lbm's run-times under TB balancing decreased in median and best-case, they increased in upper quartile and worst-case. If the tenfold increase in overhead would be a problem, the best-case run-time would not have improved. Due to the increased number of measurements, the measurement-time spikes during the MPDE cycle occur more frequent, thus affects more application run-time measurements. Consequently, the upper quartile and worst-case run-times increase. The number of repetitions must be increased to provide a better view on the run-time deviations. Additionally, the implementation must change to eliminate these spikes, as I suggested above.

Solo runs give insights into the behaviour of the algorithms and provide a base line to compare load runs to.

Concurrent Execution. As large as the gap between CFS and the other balancing algorithms sometimes is in the solo execution case, the concurrent case provides better results. All measurements in figure 5.3 are taken during the same benchmarking run of the specific balancer. Thus, the execution times of the different applications are connected, but determining if worst case time of one benchmark correlates with best case times of another, needs further analysis. Which the time budget does not allow for.

At this point, I want to recall the above-mentioned connection between outliers and the inter quartile distance in box plots.

While the performance of SLD for a single application was on a par with TB and STB, it is no competition for the two if the system is loaded. Figure 5.3 depicts the results. The inner quartile distance for Mcf, Gamess, and Lbm covers nearly the whole deviation span of the other algorithms, and for Gcc SLD shows an already large IQD and upper whisker value, but still a high number of bad performing outliers.

The median run-time is as good as the other algorithms', but SLD's worst case is more than two times worse than its median. Furthermore, in all benchmarks the distance between median and upper quartile is much larger than the distance between lower quartile and median. This asymmetry displays the unpredictability of the SLD placement. In summary, SLD provides non-deterministic performance for loaded systems with a high risk to generate a suboptimal thread-to-core mapping.

CFS on the other hand, performs well, but it is not as superior as during the solo benchmarks. Regarding minimum and maximum run-times, it performs better than TB and STB in case of Gcc and Lbm and equally well for Mcf and Gamess. Gamess has a lot of outliers, which partly can be accounted to the computation of box plots. However, the marks above 75 seconds are more than 1.5 times slower than the median run-time, which points to other issues with this workload.

I do not consider the printed outliers in case of Gcc as a problem, as they are within a 1.5 times median run-time range and compared to the other algorithms, they are below the upper quartile run-time of STB and TB. However, CFS is not the focus of this thesis, hence, I do not further investigate its issues with Gamess and Gcc.

The focus of this thesis lies in behaviour awareness, which the STB and TB algorithms implement. In difference to SLD, both perform a MPDE cycle every balancing interval. The interval length in figure 5.3 is 100ms, whereas figure 5.4 shows a 10ms interval.

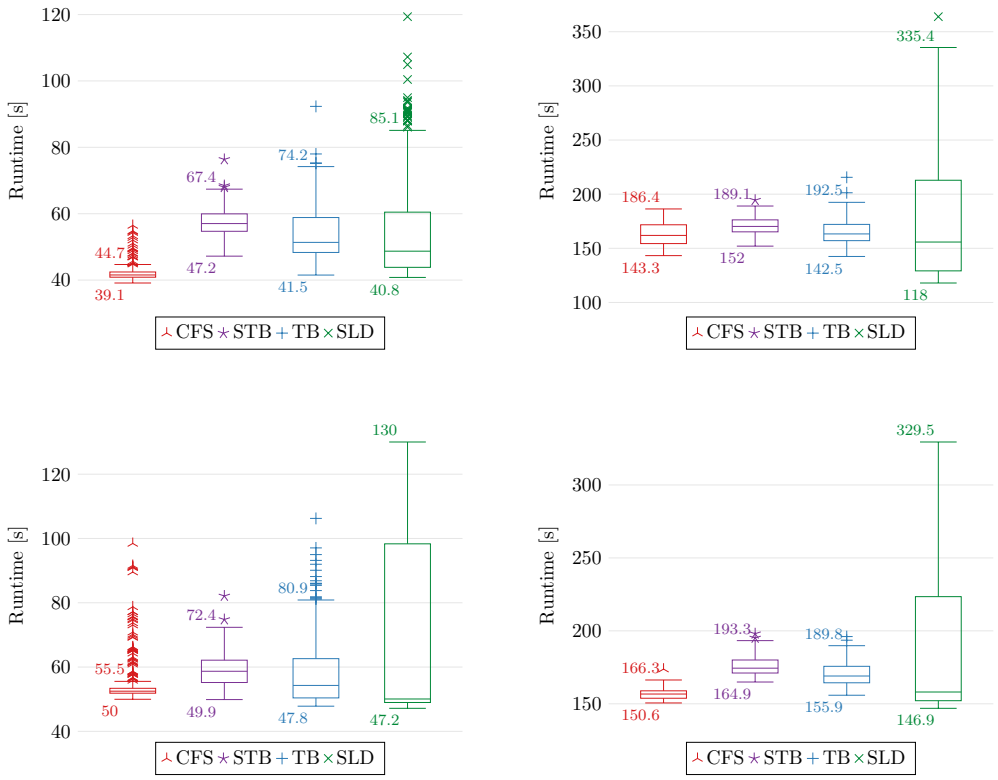


Figure 5.3: Concurrent run-times visualized using one box plot for each balancing algorithm; STB and TB use a 100ms balancing interval and all time measurements are taken from the same concurrent run. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm.

In both figures, TB's Gamess shows a behaviour similar to CFS, while the outliers in the 10ms case are less and not as high. Considering all benchmarks, TB performs better with a short interval duration: less deviation, lower worst-case execution times, and the memory-bound applications also show better best-case run-times.

STB offers more of a mixed picture: less deviation as well, but the median run-time slightly increased for all applications along with an increase in worst-case run-times. However, when STB and TB are compared regarding deviation of all benchmarks, STB displays more reliable results. STB provides each application with a fair share of resources depending on the application's run-time behaviour. As the measurements of all applications were taken during the same run, the best-case run-time of one is possibly connected to the worst-case run-time of another, hence a fair load balancer provides an equal share of resources during each execution.

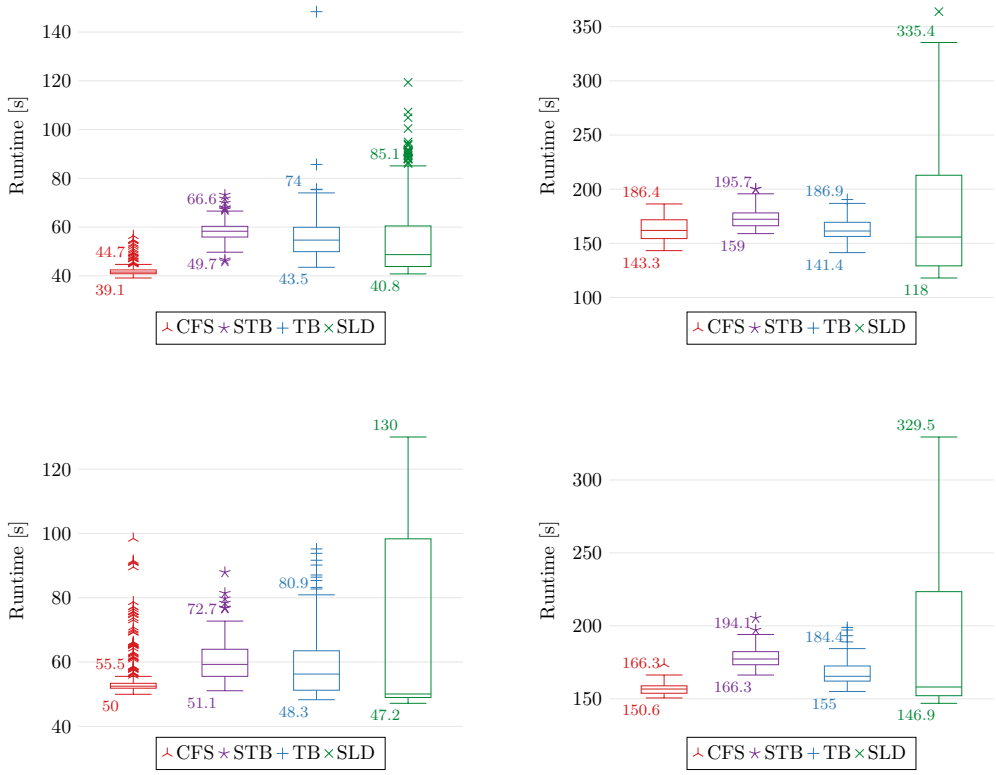


Figure 5.4: Concurrent run-times visualized using one box plot for each balancing algorithm; STB and TB use a 10ms balancing interval and all time measurements are taken from the same concurrent run. From top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm.

Heavy Load. On a quad-core machine, the four SPEC benchmarks are sufficient to fully utilize the processor. However, the quad-core machine with HT has still some underutilized resources, thus I double the workload by running each benchmark twice. [ZBF10] also turns to this approach to fully utilize the Xeon server processor with eight physical cores, where two cores share one LLC each.

Figure 5.5 shows the results for a 100*ms* load balancing interval for STB and TB. SLD is not part of this measurement, as the performance shown under less load is not competitive.

Although I run two instances of each benchmark, I only use the numbers of one instance. The results in figure 5.5 are consistent with figure 5.3 in the following two ways: First, the run-time deviation is much lower for STB compared to TB; second, the median run-time is better in case of Gcc, but worse for the other three applications. However, the worst-case performance of

STB is consistently better. It is not evident from the bottom-left plot whether STB is better; the execution time of the top mark is 373.0s.

Because the execution times of all applications are connected, the worse best-case performance of one application benefits the better worst-case performance of the other. It is preferable to improve on the median execution time of all benchmarks and I believe there is room for algorithmic improvements. However, a high worst-case execution time in one long running application like Mcf or Lbm, allows for a several good run-times in the short running Gcc or Gamess; Lbm's median is three times as high as the Gcc's or Gamess's median. So, the goal besides the numbers is to provide an environment where each application uses as much resources as needed to perform well, while not causing severe performance degradation to other applications.

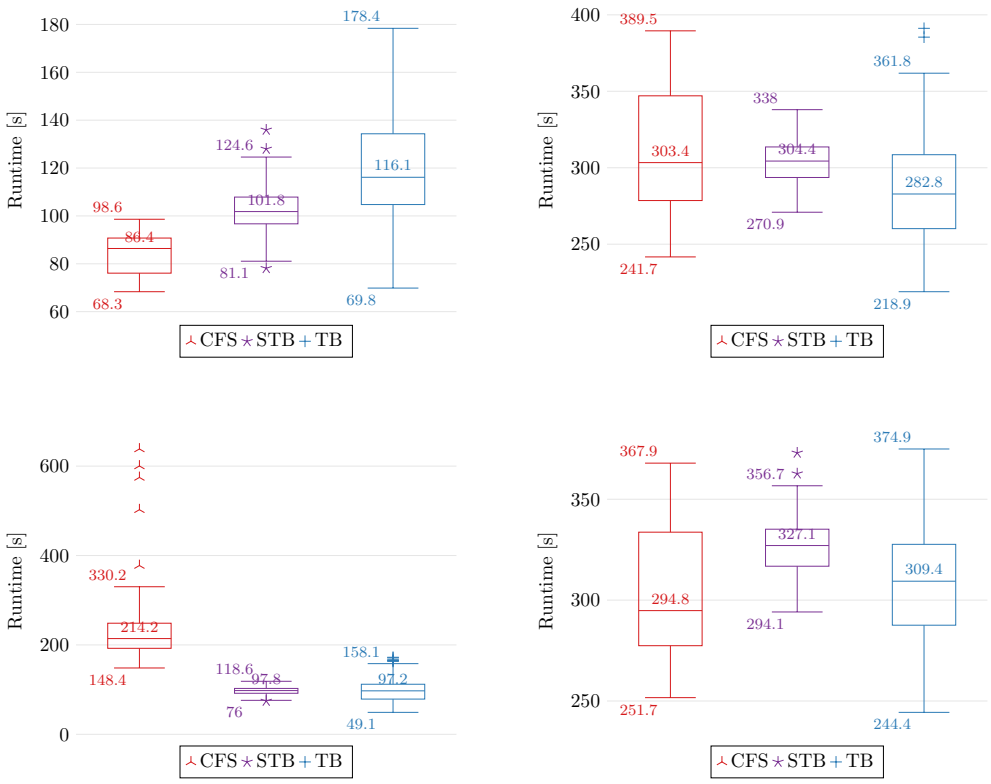


Figure 5.5: Concurrent run-times visualized using one box plot for each balancing algorithm; from top left to bottom right: SPEC Gcc, Mcf, Gamess, and Lbm. Time measurements are taken from the same concurrent run.

Overall Performance. I compute the overall performance of each load balancing scheme as average run-time increase relative to average solo run-time. Table 5.1 shows the average run-time increase per application per balancer. One group of bars in figure 5.6 displays the differences between all balancers per application. On average CFS and TB perform equally well, whereas TB is 0.06% slower. However, in the high load scenario CFS's performance is due to the performance of Gamess significantly worse than TB's or STB's. In contrast to the box plots where outliers are clearly marked, these numbers do incorporate all measurements, including the extreme run-times of Gamess under CFS displayed in figure 5.5. They also do not reflect the run-time deviation across multiple repetitions.

	CFS	STB	TB	SLD
Gamess	1.19	1.29	1.27	1.54
Gcc	1.80	1.87	1.76	1.88
Lbm	1.20	1.30	1.26	1.45
Mcf	1.91	1.89	1.82	2.08
Overall	1.53	1.59	1.53	1.74

	CFS	STB	TB
Gamess	5.02	2.12	2.11
Gcc	3.56	3.34	3.86
Lbm	2.37	2.43	2.27
Mcf	3.66	3.35	3.16
Overall	3.65	2.81	2.85

Table 5.1: Average increase in run-time per application and overall; for normal load on the left and high load on the right.

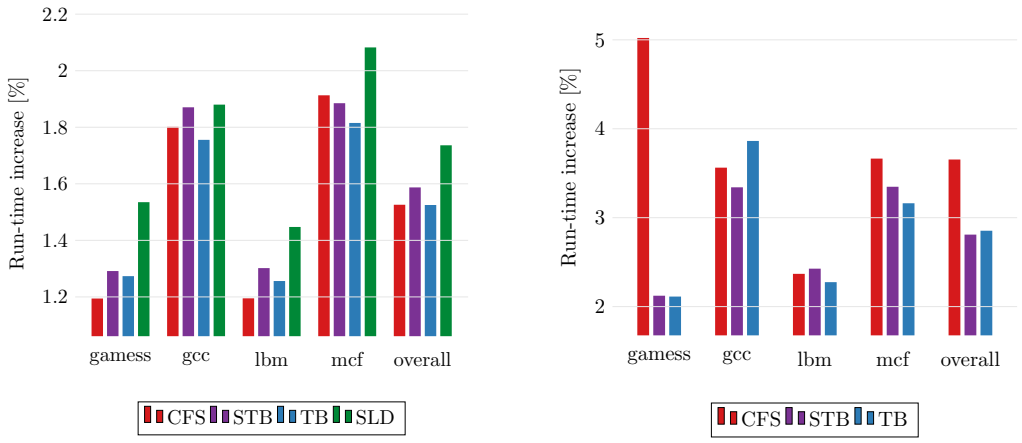


Figure 5.6: Visualized difference between the balancing algorithms per application and overall; for normal load on the left and high load on the right.

5.3 Summary

The evaluation shows that topology- and behaviour-aware load balancing has benefits regarding fairness, run-time deviation, and worst-case run-time. The solo execution baseline shows issues with measurement spikes of the behaviour-aware implementation. However, as the system load increases, space-time-balancing shows less deviation and better worst-case execution times, which suggests fair distribution of execution resources among all managed applications. Both time-balancing and space-time-balancing are superior in terms of deviation and worst-case run-time compared to the simple-load-distribution approach that does not adapt to changing circumstances or application behaviour.

The evaluation also raised issues of the implementation. The measure-predict-decide-enforce cycle shows high run-time spikes, which significantly slow down the executing applications. Besides these spikes, the MPDE cycle overhead depends on the number of managed threads and a reasonable duration of around 170µs for ten managed threads.

Some questions were not answered during this evaluation. The impact of HT-aware balancing remains an open question. Nevertheless, different implementation stages showed, that bad HT balancing visibly impacts the performance. The theory of a positive effect of communication groups was not supported with empirical data, as well as the performance of other load balancing algorithms like MPC-IPC-Placement (section 4.4).

The following section explores different directions for future work with this implementation and on this topic in general.

6 Future Work

After evaluating the benefits and constraints of behaviour-aware load balancing, I present different opportunities for improvements. This thesis provides a basis for further experiments, be it to improve isolation between task groups, expansion to multi-socket systems, or heterogeneous architectures.

Precise Event Based Sampling (PEBS). PEBS is a feature on Intel-based processors that allows the user to register an interrupt coupled to a specific counter. If the counter is increased by an event or overflows, the interrupt triggers and allows to read the hardware state from a prespecified memory region. This feature helps a developer to pinpoint, for example, memory accesses which resulted in a cache miss.

PEBS overflow interrupt could eliminate the overflow check in the thread accounting code path of the kernel. The interrupt handler must then increase the account of the currently running thread by the maximum counter value and clear the overflow flag.

Intel® Cache Allocation Technology (CAT). The most promising improvement on Intel processors is CAT ([Int15a]). It allows the user to partition the LLC or, more precisely to decide which core is able to write to which cache lines in the LLC. However, the core can still read the entire cache. This feature allows to configure the system, such that security or latency critical applications run on an exclusive core, which uses a portion of the LLC no other core has write access to. Only the core with write access to this memory area can evict cache lines.

I expect this feature to eliminate the cache as a side-channel for PRIME+PROBE attacks. Even if the attacker application runs on the same processor and shares the same LLC, it cannot write to the attacked memory. Also, real-time applications can now rely on a set of cache lines being present, if the application does not evict the lines itself.

The load balancer I present in this thesis can now not only provide temporal separation, but with the help of CAT it can also provide spatial isolation on shared hardware.

However, this hardware feature is currently only present in Broadwell or later generation Xeon™ server CPUs.

Intel® Cache Monitoring Technology (CMT). CMT allows the monitoring of individual or a group of threads, applications, or virtual machines by using a resource monitoring ID

(RMID)[1]. It monitors the cache usage of all objects with the same RMID and provides the values via control and data MSRs[2]. As this feature measures actual cache usage, I expect it to provide better information for spatial balancing than LLC misses, but LLC misses still indicate the load on prefetchers and the memory interface, which is not displayed by CMT.

However, this feature is only available in Haswell or later generation Xeon™ server CPUs.

Multi-Socket Systems. Besides CAT, usage of multi-socket systems can also prevent PRIME+PROBE attacks or provide a real-time task group with an environment with less interference. Currently, the load balancer does not support multiple sockets. First, the CPUID based topology analysis must be extended to support multiple packages and second, the core accounting must be adapted to use this new topology.

Heterogeneous Architectures. A more complex extension is the support for heterogeneous architectures, like ARM® big.LITTLE. Sarma et al. ([Sar+15]) use a predefined matrix to predict the performance change of a thread, if it is migrated to another core type. However, the authors do not consider communication or isolation realtionships. Additionally, thread groups raise numerous questions: Which core type do isolation groups need? If an isolation group is migrated to another core type, how large is the performance impact to the rest of the system? Which core type uses a server task with multiple distributed clients?

The energy consumption is also very different compared to an x86 system. If performance per energy is important, is it better to run `distr` groups constantly on the little core type?

Behaviour Categories. An approach I did not explore during this work are behaviour categories. Consider four categories: very high MPC, high MPC, low MPC, very low MPC. Each thread belongs to one category and the algorithm selects threads of different categories to execute on the same core; the SMT abstraction can also use these categories. The assignment of threads to categories could also take a behaviour history into account and categorize each thread regarding its past behaviour.

I also expect behaviour categories to help decide on difficult questions arising from heterogeneous architectures.

Hierarchy of Load Balancers. I discuss hierarchical load balancing in chapter 3.5 and decided to implement a central component. The logic in the central component does not allow to implement a balancer on top of it, which follows different balancing policies. Currently, all those decisions would be overruled. Therefore, the central load balancer approach contradicts

[1] `https://software.intel.com/en-us/blogs/2014/12/11/intel-s-cache-monitoring-technology-software-visible-interfaces`

[2] Model Specific Register

the general L4Re idea, to implement different policies on top of each other. A future project can explore designs allowing a hierarchy of balancers and evaluate the performance differences compared to the central load balancing I present in this work.

Conclusion.

This thesis demonstrated the benefit of topology- and behaviour-aware load balancing. It explored static and dynamic information sources to enable informed balancing decisions. The designed load balancer connects hardware topology with balancing on different levels, while maintaining communication relationships and isolation properties. The implementation proved the positive effect of informed decisions, through reduced run-time deviation in all applications running concurrently. Fair distribution of resources renders the system's throughput predictable for known task sets.

While the evaluation lacks an analysis of all information sources, their theoretical benefit was argued for. The evaluation also uncovered implementation deficits which disturb the measurements of heavily loaded systems. I provided ideas to improve on these deficits. However, the results present a clear tendency for behaviour-information-based load balancing to decrease influences between threads and provide each thread with a requirement-based resource share.

This thesis challenges future research to improve on cache isolation and usage analysis, but also to explore load balancing on heterogeneous hardware, where performance prediction is of even greater importance.

Bibliography

[BLS12] Daniel J. Bernstein, Tanja Lange and Peter Schwabe. 'The security impact of a new cryptographic library'. In: *Progress in Cryptology–LATINCRYPT 2012*. Springer, 2012, pp. 159–176.

[BPA08] M. Banikazemi, D. Poff and B. Abali. 'PAM: A novel performance/power aware meta-scheduler for multi-core systems'. In: *High Performance Computing, Networking, Storage and Analysis, 2008.* International Conference for High Performance Computing, Networking, Storage and Analysis. Nov. 2008, pp. 1–12.

[Cor04] Jonathan Corbet. *Scheduling Domains*. `https://lwn.net/Articles/80911/`. [Online, accessed 09-February-2016]. 19th Apr. 2004.

[Cru+14] Eduardo H. M. Cruz et al. 'Dynamic thread mapping of shared memory applications by exploiting cache coherence protocols'. In: *Journal of Parallel and Distributed Computing* 74.3 (Mar. 2014), pp. 2215–2228.

[EMR15] Stijn Eyerman, Pierre Michaud and Wouter Rogiest. 'Revisiting Symbiotic Job Scheduling'. In: IEEE International Symposium on Performance Analysis of Systems and Software. 30th Mar. 2015.

[FBZ10] Alexandra Fedorova, Sergey Blagodurov and Sergey Zhuravlev. 'Managing Contention for Shared Resources on Multicore Processors'. In: *Commun. ACM* 53.2 (Feb. 2010), pp. 49–57.

[HIB10] Steven Hofmeyr, Costin Iancu and Filip Blagojević. 'Load Balancing on Speed'. In: *Proceedings of the 15th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. PPoPP '10. New York, NY, USA: ACM, 2010, pp. 147–158.

[HPA12] John L. Hennessy, David A. Patterson and Krste Asanović. *Computer architecture : a quantitative approach*. 5. ed. Amsterdam ; Heidelberg [u.a.]: Elsevier, Morgan Kaufmann, 2012.

[Ime+15] C. Imes et al. 'POET: a portable approach to minimizing energy under soft real-time constraints'. In: 2015 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). Apr. 2015, pp. 75–86.

[Inc+15] Mehmet Sinan Inci et al. *Seriously, get off my cloud! Cross-VM RSA Key Recovery in a Public Cloud*. IACR Cryptology ePrint Archive, 2015.

[Int09] Intel Corporation. *Performance Analysis Guide for Intel® Core™ i7 Processor and Intel® Xeon™ 5500 processors*. `http://software.intel.com/sites/products/collateral/hpc/vtune/performance_analysis_guide.pdf`. [Online, accessed 11-February-2016]. 2009.

[Int15a] Intel Corporation. *Improving Real-Time Performance by Utilizing Cache Allocation Technology*. `http://www.intel.com/content/dam/www/public/us/en/documents/white-papers/cache-allocation-technology-white-paper.pdf`. [Online, accessed 09-February-2016]. Apr. 2015.

[Int15b] Intel Corporation. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. Sept. 2015.

[Int15c] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual. Combined Volumes: 1, 2A, 2B, 2C, 3A, 3B, 3C and 3D*. Sept. 2015.

[Kna+08] R. Knauerhase et al. 'Using OS Observations to Improve Performance in Multicore Systems'. In: *IEEE Micro* 28.3 (May 2008), pp. 54–66.

[LH11] Etienne Le Sueur and Gernot Heiser. 'Slow Down or Sleep, That Is the Question.' In: *USENIX Annual Technical Conference*. 2011.

[Liu+15] Fangfei Liu et al. 'Last-level cache side-channel attacks are practical'. In: *IEEE Symposium on Security and Privacy*. 2015, pp. 605–622.

[McC96] W.F. McColl. 'Scalability, portability and predictability: The BSP approach to parallel programming'. In: *Future Generation Computer Systems* 12.4 (1996). Parallel Computing Applications, pp. 265–272.

[Sar+15] Santanu Sarma et al. 'SmartBalance: a sensing-driven linux load balancer for energy efficiency of heterogeneous MPSoCs'. In: *Proceedings of the 52Nd Annual Design Automation Conference*. DAC '15. San Francisco, California: ACM Press, 2015, 109:1–109:6.

[SMa08] Claudio Scordino, Ingo Molnar and et al. *CFS Scheduler*. `https://www.kernel.org/doc/Documentation/scheduler/sched-design-CFS.txt`. [Online, accessed 09-February-2016]. 2008.

[ST00] Allan Snavely and Dean M. Tullsen. 'Symbiotic Jobscheduling for a Simultaneous Mutlithreading Processor'. In: *SIGPLAN Not.* 35.11 (Nov. 2000), pp. 234–244.

[YB14] Yuval Yarom and Naomi Benger. 'Recovering OpenSSL ECDSA Nonces Using the FLUSH+ RELOAD Cache Side-channel Attack.' In: *IACR Cryptology ePrint Archive* 2014 (2014), p. 140.

[ZBF10] Sergey Zhuravlev, Sergey Blagodurov and Alexandra Fedorova. ‘Addressing Shared Resource Contention in Multicore Processors via Scheduling’. In: *Proceedings of the Fifteenth Edition of ASPLOS on Architectural Support for Programming Languages and Operating Systems*. ASPLOS XV. New York, NY, USA: ACM, 2010, pp. 129–142.

[Zhu+12] Sergey Zhuravlev et al. ‘Survey of Scheduling Techniques for Addressing Shared Resources in Multicore Processors’. In: *ACM Comput. Surv.* 45.1 (Dec. 2012), 4:1–4:28.